Client internals and version profiles, part 2 of 4

Client internals and version profiles, part 2 of 4

This part contains 54 notes, from `client-internals/decomp-findings-2026-10-25-e/README.md` through `client-internals/gameboy-demo.md`.

`client-internals/decomp-findings-2026-10-25-e/README.md`

Decompile mining, batch e: file-transfer request lists and save rules, 2026-10-25 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code, no paths. The raw exports stay private. Nothing was pushed to Graal_analysis, no public server was contacted, the lab was not touched. The lab facts quoted are from the 2026-10-24 captures that are already summarised in earlier notes.

Scope: what the real clients keep, what they re-request, and when a local file counts as up to date (Seihai queue item 9). Read in the 2.05 rev 2, 2.17.1, 2.22 and 2.31 clients; the 2.00 client only for the list code. Checked against the 2.17, 2.22 and 2.31 login and gmap captures.

Files

file	content
01-request-lists.md	the three name lists, the window of four, which loaders ask for what, how replies settle names
02-save-rules.md	whether and where a download is written, the folder table, the time of the saved file, name lookup
03-open-points.md	what was not resolved and what would settle it
04-name-index-case-link.md	which copy of a duplicated name is used, the A–Z case fold, and the packet 40 link (a horse does not change it)
request-rules.tsv	the request-side rules, one per row, with builds and evidence
name-index-rules.tsv	the index, fold and packet-40 link, one row each
download-folders.tsv	the save-folder decision table per build
open-questions.tsv	unresolved points

Top facts

1. A client holds three name lists: wanted, update-asked, settled. A file the client lacks is asked for with `PLI_WANTFILE`; a file it has is asked about with `PLI_UPDATEFILE` carrying the file's last-write time in Unix seconds.
2. Four requests of each kind are on the wire at once. More names wait in order; a reply for a name among the first four releases the name now in fourth place. The two kinds count separately. Identical in 2.00, 2.05 rev 2, 2.17.1, 2.22 and 2.31, and seen in the lab (four and four before the first reply).
3. A local file is checked once per name per connection. Any reply (file, up to date, send failed) puts the name in the settled list; a name in that list is not asked about again.
4. The client never compares times itself. The server decides, and answers with the file, "up to date" (45, 2.1+) or "send failed" (30, which before 2.1 also means up to date).
5. Image and animation loaders: local file means use it and send the check; none means ask for it. Sound loader: ask only when missing. Levels use no file transfer.

6. Downloads are kept as cached streams and written to disk only when the `saveimages` option is on (default on). A file the client already has is replaced where it is. A new file goes to `sounds`, `maps` (2.17.1 and later) or a folder under `levels` chosen by the name.
7. The saved file gets the server's time (2.1+) so the next check carries it. A lab 2.22 run saved a downloaded gmap under `maps` with the server's time.
8. A name the server could not send is not asked for again on the same connection (lab, 2.22).

`client-internals/decomp-findings-2026-10-25-f/01-section-shapes.md`

Section shapes as seen in real GR-V1.03 files

After the board bit stream (padded to a whole byte) the file is, in order:

1. Links: one text line each, ended by a line holding only `#`. In every file the link section is `n` link lines plus `#\n`; a level with no links is just `#\n`.
2. Baddies: records of three raw bytes (`x`, `y`, type) plus a verse line ending in `\n`; the section ends with `FF FF FF` plus `\n` (the terminator is an empty verse line). Verses are separators only (`\\` seen as two backslashes for empty verses).
3. NPCs: one line each (`x+32`, `y+32`, `image#script`, script line breaks as `0xA7`), ended by a line `#`.
4. Chests: four bytes each (`x+32`, `y+32`, `item+32`, `signindex+32`) plus `\n`, ended by a line `#`. A level without chests still has the bare `#\n`.
5. Signs: records until end of file. **When a level has no signs the file simply ends after the chest `#\n`:** no sign record and no empty line. 2305 of 3113 files have no signs; 2308 files end with `#\n`.

Nothing follows the signs in any file: no `HEIGHTS`, no unknown trailing bytes (0 bytes left over in all 3113).

All counts of whole-tile coordinates check out against the board: in one supplied level a link's two tiles hold the same tile id (a door), and in the other the chest sits on its own tile id. NPC and baddy bytes are single bytes and may lie outside 0..63 (165 NPCs: x from -32 to 223, y from -11 to 223; 3 baddies with y = 255).

NPC image field: 10735 `.gif`, 1164 `.png`, 46 `.mng`, 1 `.mid`, 1 `.mp3`, and 2913 empty (script-only NPCs).

`client-internals/decomp-findings-2026-10-25-f/02-link-arrival-expressions.md`

Link arrivals are not only numbers or playerx/playery

`level-binary.md` lists a link as `dest x y w h newx newy` and the notes allow `playerx` / `playery` for the arrival. In the archive the last two fields hold, over 18,710 coordinate fields:

Form	Count
plain number (integers and half tiles such as `24.5`)	12035
`playerx` / `playery`	6652
`playerx` or `playery` followed by a signed number (`playery+10`, `playerx-15`, `playery-1.5`, `playerx+15`)	21 (in 14 levels)
a stray non-numeric word (one level wrote two such words)	2

Open: whether the stock client evaluates the offset, reads it as a number (0), or takes only the keyword. No capture shows it. The two level names that end in neither `.graal` nor `.nw` (`.enigma` 10, none 2) are also unexplained.

Link destination extensions: `.graal` 9203, `.nw` 140, `.enigma` 10, none 2.

`client-internals/decomp-findings-2026-10-25-f/03-baddy-terminator.md`

Baddy section: only a first byte >= 0x80 ends it

Two levels have a baddy at x = 0 (one at (0, 0), one at (0, 62) in a row of 16 baddies spaced 4 tiles apart ending with x = 60). `level-binary-client.md` says the section ends at a first byte of 0x80 or more; that is right. The reference decoder `formats/grv1-decoder/grv1.py` also stops at x = 0, which loses the x = 0 baddy and any after it (3 baddies in these two files) and then reads the rest of the baddy bytes as NPC lines, so NPC counts are wrong in those two files (26 against 25, 12 against 10 on the NPC count).

Corpus baddy totals: 902 baddies in 99 levels; types 0 to 9 all occur. A record's three bytes can be 0x0A or 0x00; the reader must look for the line end only after the first three bytes.

`client-internals/decomp-findings-2026-10-25-f/04-sign-and-chest-details.md`

Signs and chests in the corpus

- 2381 signs in 808 levels; 684 sit at (0, 0) (dialogue text kept by index, as `level-binary-client.md` guessed).
- 498 `#K(n)` escapes with n not 13, the commonest n values being 95 (_), 164, 61 (=), 42 (*), 36 ($), 64 (@), 94 (^), 124 (|), 134, 233, 169, 43 (+). Characters missing from the 97-entry table are written this way, as `sign-text.md` says.
- 375 chests in 88 levels. Item indexes seen: 0 to 11, 16, 17, 19, 24, and one 78 (outside the 25-name table). 343 of the chests have sign index -1 (byte `0x1F`), which is "no sign".
- The old Seihai decoder (`char + 89`) lost punctuation and line breaks; it is replaced by the alphabet decoder.

`client-internals/decomp-findings-2026-10-25-f/README.md`

Real GR-V1.03 levels: what two supplied files and a 3113-level archive show

Date: 2026-10-25. Derived facts only: counts, offsets and structure. No level content, names or paths.

Material: two single `GR-V1.03` levels, and an old private player-world archive with 3141 files named `.graal` (27 empty, 1 text `GLEVNW01` level under a `.graal` name, 3113 binary `GR-V1.03`, 2812 distinct by CRC-32). Every binary level was decoded with the rules in `formats/level-binary.md` and `formats/level-binary-client.md`, using a second implementation (Seihai's reader), and checked against `formats/grv1-decoder/grv1.py`.

Verdict on the existing notes

Note	Result
13-bit codes for `GR-V1.03`	Confirmed. 1998 of 3113 boards use a tile id of 2048 or more; 12 bits cannot carry them.
Board rules (flag bit, low-byte count, `0x100` double, runs up to 64)	Confirmed. All 3113 boards give exactly 4096 tiles; the next section starts on the next byte. 2091 files use double runs, 1809 a run of 64, none a longer one.
Section order and terminators	Confirmed (see 01).

Note	Result
Baddy section ends at first byte >= 0x80	Confirmed; `grv1.py`'s extra "or x = 0" is wrong (see 03).
Sign alphabet, one table, `byte - 32`	Consistent; 2381 signs decode, 498 use `#K(n)` escapes (see 04).
GR-V1.04 `HEIGHTS` block	No file has one; nothing to confirm.
Link arrival "number or `playerx`/`playery`"	Incomplete (see 02).

Files

- `01-section-shapes.md` section layout as seen, the no-sign-section ending, coordinate units.
- `02-link-arrival-expressions.md` arrivals written as `playerY+N`.
- `03-baddy-terminator.md` x = 0 baddies and the `grv1.py` rule.
- `04-sign-and-chest-details.md` sign escapes, chest sign index -1, out-of-range positions.
- `corpus-stats.tsv` aggregate counts.

`client-internals/decomp-findings-2026-10-25-g/01-id24-image-list.md`

Id 24 (`PLI_SHOWIMGPLAYER`) and its relay (`PLO_SHOWIMG`, 32)

Read in the 2.17.1, 2.22 and 2.31 clients. The sender routine is the same in all three (same structure, same constants); the receiver was read in 2.31.

What the text is

A list of the images that the sender's weapon scripts currently show (`showimg` and the `changeimg*` commands), one record per image. A record starts with a byte that is 0x2a plus the image's index (0 to 199). Fields follow, each a tag byte (0x20 plus the tag number) and its data. See `image-text-format.tsv`.

A tag byte of 0x2a or more starts the next record, so tags run 0 to 8. An unknown tag skips its one data byte. A tag byte with no data byte after it ends the whole text.

The first byte of a text may be the marker 0x29 (a closing parenthesis). As a GCHAR that is the value 9, which is the one byte every real client was seen to send. It means "this is the full list"; here the list is empty.

Sender

- It collects the images of the player's weapons (index 0 to 199) and compares them with the list it sent last. Changed images are sent with the fields that changed; an image that is gone is sent with an empty name.
- A counter runs from 100 down; at a wrap the text starts with the marker and every image goes out in full. The wrap is the 101st call of a periodic handler. The call after a send is skipped, so a wrap is 102 ticks. The lab period of about 5.1 s fits a tick of 50 ms; that interval was inferred from the period, not read from the client's form resource.
- Nothing is sent when the text is empty. With no images the only text that ever exists is the marker alone at a wrap, one byte: GCHAR 9. The first tick after login is a wrap.
- The client sends a UDP copy when it can (the 2.05 UDP id 15 form for older builds). The TCP copy goes out only when its test says so.

The TCP test

- 2.31: the test is a constant true. The TCP copy goes out at every wrap, alone or not, in any level.

- 2.22 and 2.17.1: the same test in both. It is true when UDP does not reach some player in the client's level; with nobody else in the level there is nothing to find and it is false. This fits the lab: 2.22 sent only with a second player present (no sends in three solo sessions), and the 90 s solo 2.17 capture saw no id 24.
- A wrap that finds the test false is lost. The next wrap is a full period later.

Receiver

- The relay is `GSHORT sender id`, then the sender's text unchanged (GServer-v2 does not look inside it).
- The 2.31 handler reads the sender's id, finds that player in the client's active level and hands the rest to the routine that applies the text. Nothing else reads the byte.
- Per record the image is looked up by index. Name: an empty name removes the image; a non-empty name creates it if missing (defaults: layer 1, colours 200 each, zoom 10, mode 0) and sets the name. Other fields are applied only to an image that exists.
- Part: 0x20 means the whole image. 0x21 is followed by left and top as two 7-bit bytes each, then width and height as one byte each; the values are clamped (left and top 0 to 16000, width and height 0 to 220).
- After a marked text, every image of that player that the text did not name is removed.

What is not known

- The sender's own value formatting for each field. The units above are the receiver's reading, so the units of x, y, colours and zoom are unconfirmed.
- What the per-player "UDP reaches this player" flag in the 2.22 test is and how it is set.
- How the client draws the images (layers, the screen-unit mode of layer 10 and over, colour and zoom).
- 2.05 uses the same tagged form (tags 0 to 6) with a 60-call full-list countdown: `../id24-205.md`. Builds before 2.05 were not read.

`client-internals/decomp-findings-2026-10-25-g/02-id46-level-switch.md`

Id 46 (`PLI_ENTERLEVEL`)

Read in the 2.31 client; 2.17.1 and 2.22 were checked for a send site and have none.

- One send site. It is at the start of the routine that switches the client's current level, whenever the client is connected, before anything else the routine does. This fits the lab: one 46 at login, one per level change, and in a gmap one per edge crossing, always before the props frame that carries the new cell prop.
- The payload is the one-character text GCHAR 1. It is a constant in the code. No level, player count, connection or option value goes into it. GServer-v2's comment guessing "a player count for the gmap level" is not what the client sends.
- The server side in GServer-v2 is a stub that does nothing.
- 2.17.1 and 2.22 have no send site for id 46. This agrees with the lab (neither sent it) and with the version profiles (46 is added in the 2.3 step).

Not resolved: what a real service does with it. Stock GServer-v2 does nothing, so no capture against it can show a reaction.

client-internals/decomp-findings-2026-10-25-g/03-id178-comma-text.md

Id 178 (PLO_SERVERWARP)

Read in the 2.17.1, 2.22 and 2.31 clients (the handler, the text splitter and the connect routine were compared and match in all three).

The handler

Build a string list from the text with the client's comma-text setter, require at least 4 items, and call the "connect to another server" routine with items 1, 2 and 3. Item 0 is not read. Anything shorter does nothing (lab: no reaction, connection stays open).

The splitter

The same code as the standard list class of the toolchain the clients are built with. Rules in `comma-text-rules.tsv`. The ones that matter for 178:

- White space (bytes 1 to 0x20) before an item is skipped.
- A quoted item ends at its closing quote; two quotes in a row are one quote; an unclosed quote runs to the end of the text.
- An unquoted item ends at a byte of 0x20 or less or at a comma. A space inside an unquoted name therefore ends the item and the rest of the name starts the next one.
- After an item: white space, then one comma, then white space. A comma at the very end adds no empty item; an empty item between two commas is kept.
- A NUL ends the text.

Consequence: a server name that contains a space must be quoted by the sender. `x,Main Server,h,1` reads as five items and the client would connect to the host `Server`.

The connect routine

- Item 1 is the server name. If its second character is a space the first two characters are dropped (type prefix, as in `P Name`). The result is stored as the current server's name. In 2.31 it is also put in the window caption: the caption is the name alone if the name already contains "graal" (any case), otherwise `Graal:` plus the name.
- Items 2 and 3 are stored as the host and the port, both as text first.
- The routine then reconnects to the host and port. The name is not used to connect, which agrees with the lab (a made-up name connected normally; reordered host and port failed to connect).

What the writers send

- GServer-v2: forwards the listserver's answer to `PLI_SERVERWARP` (41) as 178, unchanged, and only to clients from 2.1 on. The server's own code calls this a hack to let the new clients serverwarp.
- graal-serverlist: for a request it answers with the server name twice, the host and the port, as one comma text made by its tokenizer (each item quoted, quotes doubled). A commented-out older line in the same function builds `playerworld` plus a number, then the name in quotes, then the host and the port: item 0 is a tag of that kind there. That is source-comment evidence only.
- A real service's text has never been captured.

Seihai reads the text with the same rules (`delphi_comma_text`) and shows the name without the type prefix.

`client-internals/decomp-findings-2026-10-25-g/04-open-points.md`

Open points, batch g

See `open-questions.tsv` for the list with what would settle each. In short:

- Id 24: the sender's own field formatting (the layout above is the receiver's); a capture of a real client whose weapon shows an image would settle it. The lab cannot do this yet because the real client did not run NPC scripts there. The per-player "UDP reaches this player" flag of the 2.22 test. Drawing rules for layers. The 2.05 form is settled in `../id24-205.md`.
- Id 46: only a service that reacts to it could tell what it is for; stock GServer-v2 ignores it.
- Id 178: the real service's text (item 0 in particular), and whether the port is converted with a range check.

`client-internals/decomp-findings-2026-10-25-g/README.md`

Decompile mining, batch g: ids 24, 46 and 178, 2026-10-25 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code, no paths. The raw exports stay private. Nothing was pushed from here to Graal_analysis directly, no public server was contacted, the lab was not touched. The lab facts quoted are from the 2026-10-24 captures that are already summarised in earlier notes.

Scope: the three fields earlier notes called unknown (Seihai queue item 12): the meaning of client id 24 (relayed as 32), the meaning of client id 46, and how the clients read server id 178. Read in the 2.17.1, 2.22 and 2.31 clients (all three read for 24 and 178; 46 in 2.31, with a negative check in 2.17.1 and 2.22), plus the GServer-v2 and graal-serverlist sources.

Files

file	content
01-id24-image-list.md	what id 24 and its relay carry: the weapon-script image list, the full-list marker, the sender's rules, the receiver's rules
02-id46-level-switch.md	where id 46 is sent and why its payload is a literal
03-id178-comma-text.md	how the clients split the 178 text, what they do with each field, what the writers send
04-open-points.md	what was not resolved and what would settle it
image-text-format.tsv	the image-record text, one row per tag
comma-text-rules.tsv	the clients' comma-text splitter, one rule per row
open-questions.tsv	unresolved points

Top facts

1. Id 24 is a list of the images that the sender's weapon scripts show (`showimg` and the `changeimg*` commands), not a heartbeat and not an emote. The one byte 9 that real 2.22 and 2.31 clients send is the list's first character on its own: a "full list follows" marker with no images after it.
2. The sender keeps the list it sent last and sends only changes, except that every 101st call of a periodic handler it sends the whole list, led by the marker. The call after a send is skipped, so a wrap is 102 ticks:

the lab's 5.1 s if a tick is 50 ms (inferred from the period). With no images and no changes the only message is the marker.
3. The receiver (applies only for a player in its active level) creates an image when it gets a name, removes it on an empty name, changes the other fields only of an image that exists, and after a marked text removes every image the text did not name.
4. The 2.22 client sends its TCP copy only when UDP does not reach some player in the level; with nobody in the level that test is false. In 2.31 the test is a constant true. This explains the lab (2.22 only with a second player, 2.31 always). 2.17.1 has the same sender and test.
5. Id 46 is a content-free notice. One send site in 2.31 (none in 2.17.1 or 2.22): the start of the routine that switches the client's current level, when connected. The payload is a literal 1. GServer-v2 ignores it.
6. Id 178 is read by the clients' own comma-text reader, the same in all three builds: leading white space skipped, quotes with doubled quotes, an unquoted item ends at white space or a comma, at least 4 items, items 1 to 3 used. A name with a space must be quoted or it shifts the later items.
7. Item 1 of 178 is kept as the server's name: the first two characters are dropped when the second is a space (a listserver type prefix such as `P Name`). 2.31 shows it in the window title. It is not used to connect.
8. What a real list service sends for 178 has still never been captured. The open-source reimplementation sends four quoted items; a commented-out older line in its source shows a different shape with a tag in item 0.

`client-internals/decomp-findings-2026-10-25-j/01-record-shapes.md`

Record shapes seen in the real files

Counts are over the 139 distinct files (85 `Z3-V1.03`, 54 `Z3-V1.04`); see `format-stats.tsv`.

Layout

```
char[8]  magic      Z3-V1.03 | Z3-V1.04
bits     board      4096 tiles, 12-bit codes, run rules of GR-V1.00 (formats/level-binary.md)
text     links      `dest x y w h newx newy` lines, then a line holding `#`
binary   computers  x y type (3 raw bytes) + rest of line; ends at a first byte >= 0x80
text     signs      `x+32 y+32 text` lines until an empty line or the end of the file
```

All 139 files end with a line end. Sizes run 443 to 5369 bytes (median 1919).

Computers ("baddies")

- Zelda Online calls them computers. A record is `x`, `y`, `type`, then the rest of the line and a line end. `x` and `y` are raw tile bytes (not the +32 of signs); no coordinate is 64 or more.
- `Z3-V1.03`: the rest of the line is empty in all 124 records, so a record is 4 bytes.
- `Z3-V1.04`: the rest of the line holds three verses separated by backslashes (sight, hurt, attack). 45 of the 45 records carry the verse form, 40 of them with all three verses empty (two backslashes only).
- Types are 0 to 6 in every file. Counts over all records: type 0 29, 1 29, 2 30, 3 23, 4 33, 5 5, 6 20. The Java source names them: green soldier, blue soldier, red soldier, shot soldier, swamp soldier, jumping hare, octopus. Later builds' names for the numbers are not known.
- Two records (both `Z3-V1.03`) have `0x0A` as x or y. They are data, not a line end: a reader that finds the line end before taking three bytes ends the section there.
- The section ends at `FF FF FF` and a line end (the first byte is the test: any value of 0x80 or more). 42 files have computers, 97 have none and go straight to the terminator.

Signs

- 120 signs in 63 files. `x+32`, `y+32`, then text in the shared packed alphabet (`formats/sign-text.md`). No empty sign text.
- 86 signs contain a byte of 0x80 or more where a line break goes (as in `GR-V1`).
- 5 signs contain alphabet code 82 or 83; the Java source reads codes 82 to 88 differently (see README, open items).
- The sign list ends at an empty line or at the end of the file; every real file ends at a line end after the last sign.

`client-internals/decomp-findings-2026-10-25-j/02-links-and-arrivals.md`

Links: decimals, negative arrivals, destination names

Counts over the 139 distinct files: 338 links in 136 files (`Z3-V1.03` 207, `Z3-V1.04` 131).

Decimals

- `Z3-V1.03` links are written with decimal points: 167 of 207 have a decimal arrival, 124 have a decimal x, y, width or height (47 of the 85 `Z3-V1.03` files). Geometry is whole tiles with a `.0` ending; arrivals are often half tiles (171 links have a fractional arrival, 79 in `1.03`, 92 in `1.04`).
- `Z3-V1.04` links have no decimal geometry (0 of 131) but 92 have a decimal arrival.
- All three loaders read six floating-point numbers (the Java source with `parseDouble`-style reads, 1st Graal and 1.39 with string-to-float). A reader that parses integers drops these links.
- No destination name contains a space (0 of 338), so splitting the first token off is safe for these files. `formats/level-binary.md` says tokens before the last 6 are rejoined for GR-V1; the Delphi clients take the destination up to the first space.

Negative arrivals

- 67 links have a negative x arrival and 85 a negative y arrival (151 links in all; one has both). `Z3-V1.03` and `.04` both use them.
- Rule, from the Java source and 1st Graal: a negative arrival coordinate is a flag meaning "keep the player's own coordinate on that axis". Both set a "constant" flag from `< 0` and use the player's position otherwise.
- The same appears in `GR-V1` links: 805 of 5381 distinct links in the same builds have a negative arrival. That the 2.x clients read it the same way is inferred, not traced.

Destination extensions

- `Z3-V1.03` links name `.zelda` (205) or `.editor` (2) files. `Z3-V1.04` links name `.graal` (76) or `.zelda` (54), or `.editor` (1).
- The extension in a link therefore differs between builds that name their levels `.graal` and builds that name them `.zelda`; the loader sees the magic only (see 03).

`client-internals/decomp-findings-2026-10-25-j/03-extensions-and-magic.md`

Extensions say nothing; the magic does

Distinct files (by content) in the old builds, by magic and extension:

Magic	.zelda	.graal	.editor	other
Z3-V1.03	84	0	1	0
Z3-V1.04	23	31	0	0
GR-V1.00 to GR-V1.03	694	1271	9	111 (assorted suffixes)

- The .zelda extension is not a format marker: 694 of 801 distinct .zelda-named files are GR-V1.00 to GR-V1.03. 2nd Graal to 15th Graal ship 68 to 368 such files per build (by-build.tsv, column gr_magic_in_.zelda).
- 1st Graal's 16 levels are all Z3-V1.04 under .graal names.
- The Java Zelda Online game folder holds 51 Z3-V1.03 files, all .zelda, and no Z3-V1.04.
- The player-world archive (.graal and .nw names) holds no .zelda file and no Z3-V1 file.
- Every client binary tested (1st Graal, 1.28b2 to 1.41, 2.0 to 2.31, the two NewWorld betas) carries both Z3-V1.03 and Z3-V1.04 as magic strings; none carries a third Z3-V1 revision. The 1st Graal and 1.39 loaders were read: they compare the 8 magic bytes only. 2.0 to 2.31 were not traced beyond the strings.

---

client-internals/decomp-findings-2026-10-25-j/04-loader-rules-compared.md

Loader rules compared

Three readers were compared for the points where a re-implementation can go wrong. "Delphi clients" means 1st Graal and 1.39, which agree on every row unless stated.

Point	Java Zelda Online	Delphi clients	Real files
Code width	11 bits for Z3-V1.02, 12 for .03	12 for Z3-V1.03/.04	12 (no .02)
Control code	top bit set, low byte count, 0x100 double	same	same, 4096 tiles in all
Double flag	stays set until a double run completes	not separated	the GR-V1 reader clears it at the next control code; no file shows a difference (0 of 663)
Link numbers	six doubles	six floats	decimals present
Negative arrival	"constant" flag from < 0	same	151 links
Computer record	4 fixed bytes read as x, y, type, line end (Z3-V1.03 form)	1.39: bytes are added until a line end after 3 or more bytes; a first byte below 0x80 starts a record. 1st Graal reads by line (inferred from its shape)	all .03 records have an empty rest
0x0A as x or y	harmless (fixed bytes)	harmless in the 1.39 loader (it keeps adding bytes until the line is longer than 2); 1st Graal would end the section	2 records (not run in 1st Graal)
Verses	none	split at backslashes, three strings (.04)	.04 only
Signs	x+32, y+32	same, text byte - 32 through the shared alphabet	consistent
Sign list end	empty line or end of file	empty line	both occur

Consequences for a re-implementation:

- Read the three computer bytes first, then the line; do not look for the line end before taking x, y, type.
- Parse link numbers as floating point and treat a negative arrival as "keep the player's coordinate".
- Do not dispatch on the extension.
- The double flag rule and the sign-end rule do not matter for any real file; follow the clients.

`client-internals/decomp-findings-2026-10-25-j/05-old-reader-failures.md`

What a GR-V1 section reader gets wrong on Z3-V1

A section reader written for `GR-V1.03` (integer links, three-byte baddy records, an NPC and a chest section where `Z3-V1` has none) was replayed over the 139 distinct files. Counts:

Fault	Effect	Count
Link numbers parsed as integers	decimal links dropped	124 links in 47 `Z3-V1.03` files
Computer line end left in the stream	next record starts one byte late; the type byte of the record and the line end shift	all 31 `Z3-V1.03` files that have computers (one file: 14 computers reported, 10 present)
Negative arrival read as a literal coordinate	arrival at -1	151 links (also 805 `GR-V1` links)
`.zelda` extension treated as a promise	`GR-V1` files read as `Z3-V1`	694 distinct files under `.zelda` names are `GR-V1`

After switching to a strict reader every one of the 663 copies parses to exactly the end of the file with no leftover bytes and no anomaly.

`client-internals/decomp-findings-2026-10-25-j/README.md`

Z3-V1 (`.zelda`) levels: format checked against 139 real files and three clients

Date: 2026-10-25. Derived facts only: counts, offsets and structure. No level content, names, paths or code addresses.

Material: the old client builds' level folders (1st Graal to 15th Graal, Graal 1.1 and 1.1.9, the 2nd/4th "alpha" folders, and the Java Zelda Online game folder with its source), which hold 663 `Z3-V1` files (139 distinct by content). Each file was decoded twice, with a throwaway Python scanner built on `formats/grv1-decoder/grv1.py` and with Seihai's new strict Rust reader, and the two agreed. The Java Zelda Online source, the 1st Graal client and the 1.39 client were read for the loader rules (derived facts only; no decompile text is kept here). A private player-world archive was counted for comparison.

Verdict on the existing notes

Note	Result
Sections: board, links, baddies, signs; no NPC and no chest section	Confirmed. All 139 distinct files end exactly at the end of the signs.
12-bit codes for `Z3-V1.03` / `.04`	Confirmed (all 139 boards give 4096 tiles and the links start on the next byte).
Verse line only in `Z3-V1.04`	Confirmed, with detail: see 01. `Z3-V1.03` records have an empty rest-of-line, so every record is 4 bytes (3 + line end), as in the Java source.
Baddy terminator `FF FF FF` + line end	Confirmed.
Links `dest x y w h newx newy`	Incomplete: `Z3-V1.03` writes decimals and `-1` arrivals (see 02).
`Z3-V1.02`	Only the Java client reads it; no real file has it.
"Only GR-V1.00 to 1.03 load, `.zelda` Z3-V1 for the offline levels" (1.33 to 1.39)	Incomplete. Every Graal client binary from 1.28b2 to 2.31 carries both `Z3-V1.03` and `Z3-V1.04` magics, and the 1st Graal and 1.39 loaders accept them whatever the file name.

Files

- `01-record-shapes.md` board, link, computer and sign records as seen; computer types; `Z3-V1.04` verses.
- `02-links-and-arrivals.md` decimal links, negative arrivals, destination extensions.

- `03-extensions-and-magic.md` which names hold which magic; what the loaders key on.
- `04-loader-rules-compared.md` Java, 1st Graal and 1.39 rules side by side, with the differences that matter.
- `05-old-reader-failures.md` what a `.graal`-style section reader gets wrong on `Z3-V1` (counts).
- `by-build.tsv` per-build counts (18 builds).
- `format-stats.tsv` aggregate counts over the 139 distinct files.

Open

- (Answered later in `../bare-level-names.md`: no build tries `.zelda` for a bare name and none falls back; the editor adds `.graal` to a name saved without an extension.)
- The 2.0 to 2.31 loaders were checked for the magic strings only, not traced.
- Java sign alphabet versus the shared one for codes 82 to 88 (5 real signs touch 82 or 83; not compared).
- Computer type names beyond the Java source's 0 to 6 (none seen).
- No `Z3-V1.02` sample.

`client-internals/decomp-findings-2026-10-25-k/01-knockback-and-packet-hits.md`

Knockback and the packet hits

Knockback of a hit

The routine that hurts the player takes a power in hearts and a knockback vector (dx, dy). It sets hurt mode (mode 3, counter 40) and stores the vector scaled to length one tile. A zero vector is replaced by one tile east.

For the first five ticks of hurt mode (the counter reads 39 down to 35 after its decrement) the player moves one tile along the stored vector each tick, then the body box is tested at twelve points; if any is blocked the step is undone and the push is tried again the next tick. On open ground the player ends five tiles away after 250 ms. The test uses the same ground rules as an ordinary walk (blocking tiles, and chests).

The twelve points lie on the edges of a box 0.6 to 2.4 tiles across and 1.1 to 2.9 tiles down from (X, Y) (the position properties): three points per edge, the four corners counted twice; see `knockback-probes.tsv`. The box is centred on (X+1.5, Y+2.0). The routine and the table are the same in 2.05 rev 2, 2.17.1, 2.22 and 2.31.

Where the vector comes from: - sword hit: from the attacker's position to the victim's; - packet 40: its two bytes (the byte minus 0x60, over 16, tiles); - packet 46: the victim's centre minus the hit point.

Gates shared by the sword and the packets

Read in 2.05 rev 2 and 2.22. In this order: 1. the attacker's player id is known; 2. the victim is hurtable (not already hurt, not dead, not swimming); 3. the victim's AP is not 100; 4. the level has not run no-player-killing; 5. the attacker's once-a-second stamp: the second of the day is compared with the one stored for that attacker and stored; the table is shared by the sword and both packets, and the store happens before the guild test; 6. the guild test: the first bracket pair in the attacker's nickname must not occur in the victim's nickname; 7. for the packets, the per-player field test (below); 8. packet 40: the squared distance between the two X/Y positions is at most 100 (ten tiles). Packet 46: the box test below.

The sword path adds the reach box and an attacker status test; the packet path has **no** attacker status test.

Power: packet power is capped at one heart; a power of 0 is accepted (hurt mode and knockback, no hearts taken).

Packet 46 box

The hit point is (b1/2, b2/2) in tiles, from the two bytes after the power. The player's centre is (X+1.5, Y+2.0). A hit lands when the absolute difference is strictly under 2 on each axis.

The per-player field of packet 40

Packet 40 is dropped unless a link integer on the attacker equals the same integer on the victim. A new player is created with 0. The horse image is a different slot and does not write the integer, so a player who only has a horse still matches an ordinary player. In 2.22 the integer is the layer/parent link (layer number minus one, plus the parent's link when there is a parent). Detail: `../decomp-findings-2026-10-25-e/04-name-index-case-link.md` **[bin]**.

What stays unknown

- ~~Which object kinds (bomb, arrow, baddy) call the hurt routine with which power~~ — closed in `../damage-sources.md`.
- ~~Whether the respawn routine's second entry save lands on the death spot online~~ — closed in `../lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md` §D (stays the entry point).
- ~~Whether the real client sends CURPOWER after damage~~ — closed in the same lab note §C.
- ~~Packet 40's per-player owner/parent field with a horse or similar (still needs a capture).~~ Closed in `../decomp-findings-2026-10-25-e/04-name-index-case-link.md`: a horse does not change the link.

`client-internals/decomp-findings-2026-10-25-k/02-save-folders-by-build.md`

Save-folder layouts by build, and the empty-stream guard

Each build's save routine for a downloaded file was read (static). The table is `save-folders-by-build.tsv`.

Layouts

- **1.33 to 1.41** (1.33, 1.34, 1.35, 1.36, 1.37, 1.38, 1.39, 1.41 read): one folder for everything that is saved, `levels/webgifs`. The 1.39 routine is entered only for names ending `.gif`, `.png` or `.mng`, so no sound is saved. It deletes an existing file with the same path first, writes the stream, and sets no modification time.
- **1.42 NewMain**: the two builds differ. The earlier one adds `ganis`, `heads`, `swords`, `shields`; the later one adds `sounds` (for `.wav`) and `bomys`. Not read further.
- **2.00 (all four revisions) and 2.01 (both)**: `sounds` for `.wav`. Under `levels`: `bomys` for names starting `bomy` or `b_`, `ganis` for `.gani`, `emoticons` for names starting `emoticon`, `heads`, `bodies`, `swords`, `shields` for names containing those words, else `webgifs`. There is no `hats` folder: a `hat0.png` goes to `webgifs`. There is no `maps` folder.
- **2.02 (all three revisions), 2.03, 2.04, 2.05 rev 1 and 2, both 2.1 test builds**: the same list plus `hats` (name starts `hat`, fourth character a digit, in the builds that test it). Still no `maps`.
- **2.17.1, 2.22, 2.31**: the same, plus `maps` for `.gmap` and for `.txt`, `.gif`, `.png`, `.mng` names containing `map`.
- 2.1 to 2.16 have no binary: when `maps` arrived between the 2.1 test builds and 2.17.1 is unknown.

The guard

Every build read (1.39, 2.00, 2.05 rev 2, 2.17.1, 2.22, 2.31) begins the save routine with: the stream exists and its size is above zero. Otherwise the routine returns: nothing is written, an existing copy is not deleted, the modification time is not touched and the file is not announced to the loaders. A stream left empty by a failed send is therefore never saved.

Login request set (compared with traces)

In the 2.22 and 2.31 lab traces (real client against a real server, and Seihai in place of the client, same server) the login requests of the two clients match for the body-part images, icons and the gmap. The remaining difference is the second default-weapon image, which the real clients ask for and Seihai did not: the real order is the bomb image, then the bow image.

Evidence about the default weapons (not used to wire anything): the open server source numbers the default items with bow 7 and bomb 8; the 2.22 client's table of item names follows the same order; the real login asks the bomb image before the bow image, as a walk over default weapons 8 then 7 would. Which wire id names which image was not seen directly.

`client-internals/decomp-findings-2026-10-25-k/03-open-points.md`

Open points

- ~~Bomb, arrow and baddy hurt powers: the object-kind mapping was not traced.~~ **Closed.** Powers and kinds are in `../damage-sources.md` **[bin]** (explosion types 0.5/0.5/1, arrow 0.5, fireball 1, baddy touch 0.5 / golden warrior 1, etc.). Lab bomb/arrow blast-shape and arrow type-byte quirks remain open in `../lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md` and do not reopen the power table.
- ~~The per-player field of packet 40: inferred to be always equal for ordinary players. A real client hit by packet 40 from a player that owns a horse or similar would show whether it differs.~~ **Closed.** The integer starts at 0 and the horse image does not write it, so a player who only has a horse still matches an ordinary player. In 2.22 the integer is the layer/parent link. `../decomp-findings-2026-10-25-e/04-name-index-case-link.md` **[bin]**.
- ~~End of respawn: the respawn routine calls the entry save once more after asking for the warp; whether the next respawn point is then the death spot online needs two respawns on a server that delays the level reply.~~ **Closed.** Lab: next respawn stays the level entry point, with or without a delayed reply — `../lab-captures-2026-10-25/01-damage-chest-respawn-weapons.md` §D **[cap]**.
- ~~CURPOWER after damage: needs a real client at full hearts hit by an injected packet 40 and a look for a props message with the hearts byte.~~ **Closed.** After a hit the client sends one props message with CURPOWER (half-hearts) + `hurt` GANI — same lab note §C **[cap]**.
- ~~Default weapon ids against images: see 02.~~ **Partly closed.** Lab §E: id **7** → `wbowi1.png` settled; id **8** → `wbomb1.png` not isolated (bomb image is asked anyway). Also ids 12, 20–23 → named images. Same lab note **[cap]**.
- ~~Level cache format (GWEBL001): not part of this batch.~~ **Closed** for format and key handling: `../../formats/gwebl001-cache.md` **[2.05][test]** (144 real files). Write is triggered by server packet 39 (`PLO_LEVELMODTIME`).
- ~~File-index scan order for two files with the same name in different folders: a capture with different bytes in `levels/webgifs` and `levels/bodies` and a look at which is drawn.~~ **Closed.** Neither folder wins by rule. 2.22 keeps the newer file; 2.05 rev 2 keeps the first hit. Same note **[bin]**.

- ~~Whether pre-2.1 clients re-send the file check for a settled name: a 2.05 run against a server that answers every check with "send failed".~~ **Closed.** Settled-list rule: any reply puts the name in C; a name in C is not asked again. Same list code in 2.00 and 2.05 — `../decomp-findings-2026-10-25-e/01-request-lists.md` **[bin]**.
- When the maps folder rule arrived (2.1 to 2.16): still no binary. A 2026-11-01 listing of the client archive has the two 2.1 test builds (already read: no `maps` rule) and then 2.22, and nothing between.

`client-internals/decomp-findings-2026-10-25-k/README.md`

Decompile findings, batch k: hit knockback, packet-hit gates, save-folder layouts by build, 2026-10-25 (Asia/Tokyo)

Derived facts only. No raw decompile text, no addresses, no generated function names, no code, no paths. The raw exports stay private. No public server was contacted and no lab run was made for these facts.

Scope: the leftovers of batch d (damage and respawn) and batch e (file transfer). Static read of the 2.05 rev 2, 2.17.1, 2.22 and 2.31 clients (hit path), and of the 1.33 to 1.41, 2.00 to 2.05, both 2.1 test builds, 2.17.1, 2.22 and 2.31 clients (save routine).

Files

file	content
01-knockback-and-packet-hits.md	the knockback of a hit, the gates of packets 40 and 46, the 46 box, what stays unknown
02-save-folders-by-build.md	the save routine's folder layout in each build, the empty-stream guard
03-open-points.md	what was not resolved and which capture would settle it
knockback-probes.tsv	the body-box probe offsets
save-folders-by-build.tsv	the folder layouts per build
open-questions.tsv	unresolved points

Top facts

1. A hit starts hurt mode (40 ticks) and moves the player one tile per tick for the first five ticks along the knockback vector scaled to length one (a zero vector means east). After each step twelve body-box points are tested; if one is blocked the step is undone. Same in 2.05 rev 2, 2.17.1, 2.22, 2.31; the twelve offsets are identical in all four.
2. Packets 40 and 46 go through the sword's gates (attacker known, victim hurtable, AP not 100, level not no-player-killing, the once-a-second stamp per attacker shared with the sword and taken before the guild test, the guild tag test) and then a range test (40) or a box test (46). There is no attacker status test on the packet path. Power is capped at one heart; power 0 is accepted.
3. Packet 46's hit point is (b1/2, b2/2); a player's centre is (X+1.5, Y+2.0); the box is strictly under 2 tiles on each axis. The knockback is centre minus hit point.
4. The integer compared in packet 40 starts at 0. A horse does not change it, so a player who only has a horse still matches an ordinary player. In 2.22 it is the layer/parent link. See `../decomp-findings-2026-10-25-e/04-name-index-case-link.md`.
5. Save folders: 1.33 to 1.41 save everything in one folder (`levels/webgifs`); 2.00 and 2.01 have the folder list without `hats`; `hats` appears in 2.02; both 2.1 test builds have no `maps` folder rule, which first shows in 2.17.1.

6. Every build from 1.39 to 2.31 returns from the save routine at once for a missing or zero-length stream: nothing written, nothing deleted.

`client-internals/decomp-findings-2026-10-25-l/01-walking-body-box.md`

Walking: the wall test is a box, not a point

Date: 2026-10-25. Source: the 2.05 client's walking routine (derived).

The older note (`../foot-test.md`) measured a single-point test in an SDL reimplementation and said the official client "used a different body-block mask". This is that mask.

Step

- Each arrow key is a separate step, handled in the order up, left, down, right. A diagonal is two steps, each tested against the position the previous one left.
- A step is half a tile times a speed factor: 1.0 on foot, 1.2 with a high speed state. Other factors (swimming, carrying) exist; their values were not read here. The step vectors per facing are (0, -0.5), (-0.5, 0), (0, 0.5), (0.5, 0).

Six probe points per facing

Offsets from the sprite's top-left, evaluated at the position the step would reach:

facing	the points
up	y + 1.0; x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475
left	x + 0.5; y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975
down	y + 3.0; x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475
right	x + 2.5; y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975

That is a box x + 0.5 .. x + 2.5, y + 1 .. y + 3 (two tiles square), with the points on its leading edge. Points are truncated to a tile.

Rule

- The step is kept when the **first four** points are free.
- Otherwise the position is put back and the player is put in the "pushing" state (the push/pull block action starts from that state).
- On a refused step, a 0.1-tile nudge along the edge may apply so a corner does not catch. With points numbered 1..6: if point 1 is blocked and 3, 4, 5 are free, move +0.1 across the edge (right for up/down, down for left/right); if 1 and 2 are free, 4 is blocked and 6 is free, move -0.1.
- After the step the fractional part of the position is snapped to a tile edge when it lies within 0.0625 of it (not read in detail).

What a point tests

Blocked if any of: a chest record covers it (2 x 2, edges included, opened or not); a solid object sprite or another blocking sprite is there; the tile is neither in the passable list (`arrays.dat` array 0) nor in the liquid list (array 1). Lava is a liquid and walkable.

Other probes (same routine family)

- A-key probe, lift, and sword cut: one tile beyond the middle of the leading edge: up (x + 1.5, y), left (x - 0.5, y + 2), down (x + 1.5, y + 4), right (x + 3.5, y + 2).
- A separate probe set is used by the push/pull block action; not read here.

`client-internals/decomp-findings-2026-10-25-l/02-arrays-dat.md`

arrays.dat: layout and meaning

Date: 2026-10-25. From the client's loader and its uses (derived).

Layout

A run of arrays to the end of the file (the 2.22 file: 11,500 bytes, 25 arrays). Each array starts with a big-endian `u32` header:

- low 16 bits: the count `n`;
- high 16 bits: 0 = `n` big-endian `u32` ints follow; non-zero = `n` nested arrays follow, each with its own header.

The 2.22 file parses completely this way with no bytes left over. Counts of the 25 arrays: 934, 116, 133, 76, 40, 4, 4, 79, 4, 2, 2, 5, 5, 4, 2, 2, 2, 124, 8, 6, 32, 4, 5, 4, 356.

Arrays (index = position in the file)

index	what	shape
0	passable tile ids (`nowall`); any other tile is a wall	flat
1	liquid tile ids (water and lava)	flat
3	lava tile ids (liquids that are lava)	flat
9	cuttable 2 x 2 objects	nested: type -> column -> row
10	the 2 x 2 tiles a cut leaves	same shape, same order
11	liftable 2 x 2 objects	nested
12	the 2 x 2 tiles a lift leaves	nested
13	closed chest tiles (1784 1785 1800 1801)	flat
14	open chest tiles (829 845 / 830 846)	nested

Others (2, 4 to 8, 15 to 24) are not read here; several hold per-tile lookups the client uses for drawing and sounds.

Objects are indexed `[column][row]`. The 2.22 tables: 2 cut types (the bush `2 3 / 18 19` and `420 421 / 436 437`, leaving `677 678 / 693 694` and `679 680 / 695 696`), and 5 lift types (the bush, a 512 block, a 684 block, the rock `34 35 / 50 51`, a 990 block) leaving `677 678 / 693 694`, `1802 1803 / 1818 1819`, `1770 1771 / 1786 1787`, and `1834 1835 / 1850 1851` twice.

Note: reading only the first four arrays as flat int lists (as a first loader did) is right for 0 to 3 and ignores everything else; arrays 9 to 12 need the nested reading.

client-internals/decomp-findings-2026-10-25-l/03-cut-lift-chest.md

Sword cut, lift, and chests

Date: 2026-10-25. From the 2.05 client (derived).

Matching a 2 x 2 object

Take the tile under the probe point. For each table entry in order, for each of its four tiles equal to that tile, the object's top-left is the point minus that tile's column and row. The four board tiles there must equal the entry's four, all on the board. First full match wins.

Sword cut

At the start of a sword swing, at the A-key probe point: match against the cut table (all entries, no glove). On a match the four tiles are replaced by the entry's "left" tiles (array 10) through the routine that also writes board changes the server relays, then a routine that takes the entry index spawns debris or items (not read).

Lift (A key)

First action of the A key (checked before pulling a block). Glove power `g` allows the first `g + 2` lift entries: a bare hand the first two (bush, 512 block), power 1 adds the 684 block, power 2 the rock and the 990 block. A match replaces the four tiles with the entry's "left" tiles (array 12) and the player carries an object. The throw, its flight and landing, and the carrying pose were not read.

Chests

The open actions below are what the Seihai client does after its reading of the 2.05 client; only the lift and cut probes were read in detail.

- By board tile: if the tile at the A-key probe is one of the closed chest tiles (1800/1801 bottom row), it is replaced by the open chest 2 x 2 and PLI_OPENCHEST is sent.
- By record: a `PLO_LEVELCHEST` record at the probe, not yet opened: it is set opened, the item is spawned, PLI_OPENCHEST is sent, and the sign the record names is shown.
- A chest record has **no draw code**. A chest looks like its board tiles and nothing else. The record adds the open action and a 2 x 2 solid footprint (opened or not).
- Record bytes (each plus 32): opened flag, x, y, and, only if not opened, item index and sign index. The item index names an entry of the client's built-in chest item list (the list was not read here).

A-key order

Lift is tried first, then grab (pull a block). Where reading a sign and opening a chest sit in that order was not read.

Sword while carrying

See `../decomp-findings-2026-10-28/01-sword-while-carrying.md`: with an object carried, the sword key throws it instead of swinging.

client-internals/decomp-findings-2026-10-25-l/04-public-server-run.md

A public GServer-RS login, 2.22 profile (2026-10-25)

Counts and layouts only. The capture, screenshots, and the server's address are not in this repository.

Login burst

819 messages in the session. In order, the ones that matter: own player props, flags, a weapon list of one weapon (`-gr_movement`, with its script), `SETACTIVELEVEL`, the level board (64 x 64 tile ids, two bytes each), three `PLO_LEVELCHEST` records, no `PLO_LEVELSIGN`, four `PLO_NPCPROPS` records, then text and window messages. One file request in the session was answered with a send-failed message.

Why no chests were drawn

All three chest records are the short form (opened, x, y): the server says the account had opened them. The board has none of the chest tile ids (closed 1784 1785 1800 1801 or open 829 830 845 846) and the three positions are plain grass. The client draws nothing for a chest record, so nothing is drawn.

Weapon scripts and line breaks

The server writes a line break in a weapon script as §. A latin-1 server sends the byte A7; a UTF-8 server sends C2 A7. Decoded as latin-1 without handling the pair, the script becomes one `//` comment line and does nothing. With the break read as a newline the `-gr_movement` script parses with no diagnostics. It sets two player strings from the position, rounded to half a tile, and sets the chat property to a fixed word, every 0.05 s.

Capture link type

A capture taken with `tcpdump -i any` uses Linux cooked v2 (pcap link type 276, a 20-byte pseudo-header whose first two bytes are the protocol type); older cooked captures use type 113 with 16 bytes.

---

client-internals/decomp-findings-2026-10-25-l/README.md

Decomp findings 2026-10-25 (l): walking body box, arrays.dat, cut/lift, a public-server run

Derived facts from reading the 2.05 client (walking, lifting, sword cut, chest open) and from one login to a public GServer-RS with a 2.22 profile. No addresses, no function names, no decompiler output, no code. Numbers are tile units unless stated.

- `01-walking-body-box.md` — the wall test of the walking routine: a box with four probe points per facing, keys applied one at a time, corner nudge. Corrects `../foot-test.md`'s "official ~2.05 used a different body-block mask": that mask is this box.
- `02-arrays-dat.md` — the full layout of `arrays.dat` (25 arrays, nested kinds) and what each array is for.
- `03-cut-lift-chest.md` — sword cut, lift by glove power, chest open by tile and by record, the A-key order, probe offsets.
- `04-public-server-run.md` — what a public server sent on login (counts and layouts), why no chests were drawn, the weapon-script line-break byte, the capture link type.
- `open-questions.tsv` — what was not read.

Checked against: Seihai PRs 60 to 64 (tests listed there).

`client-internals/decomp-findings-2026-10-25-m/01-walking-rule.md`

The walking rule, whole

Date: 2026-10-25. Source: the walking routine of the 2.05, 2.17.1, 2.22 and 2.31 clients (derived; the four agree on every table and constant below).

One key, one step

The routine is given the key's direction, 0 up, 1 left, 2 down, 3 right, and runs the step for that direction. With several keys held the steps run in the order up, left, down, right, each at the position the earlier ones left. The first thing it does is set the player's facing to that direction. A step is half a tile times a speed factor (1.0 on foot); the step vectors are (0, -0.5), (-0.5, 0), (0, 0.5), (0.5, 0). Positions are doubles, in tiles.

The probe points

Six points per facing, as offsets from the sprite's top-left, read at the position the step would reach (table as in `../decomp-findings-2026-10-25-l/ 01-walking-body-box.md`):

facing	the six points
up	y + 1.0; x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475
left	x + 0.5; y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975
down	y + 3.0; x + 0.525, 1.475, 1.525, 2.475, 2.525, 0.475
right	x + 2.5; y + 1.025, 1.975, 2.025, 2.975, 3.025, 0.975

Numbered 0 to 5 in that order. Each point goes through the same blocked test, which returns blocked or free.

Point to tile. A coordinate becomes a tile as `floor(v + 0.0001)`. A point exactly on a tile line reads the tile that starts there; a value a hair under a whole number (rounding) reads the tile above it. A tile outside 0..63 is not read from this level: it is the neighbour in a gmap, or a fixed tile outside a gmap.

What blocks a point. In this order: a list of objects with two flags and a body box like the player's (only when a client option is on); the level's NPCs that have a blocking shape or image; a list of 1.5 x 2 boxes with an enabled flag (x + 0.25 to x + 1.75, y to y + 2); the chests (each record x, y to x + 2, y + 2, edges included, opened or not); then the tile, which blocks unless its id is in one of the two passable lists of `arrays.dat` (arrays 0 and 1). The first list and the 1.5 x 2 list were not identified.

Decision

Points 0 to 3 are the step test. If all four are free the step is kept (and the position is set to the stepped one). If any is blocked it is refused.

On a refused step

In this order:

1. **Corner nudge.** If point 0 is blocked and points 2, 3 and 4 are free: add 0.1 across the edge (+x for up and down, +y for left and right). Else if points 0 and 1 are free, point 3 is blocked and point 5 is free: add -0.1.

The nudge table is the same in all four clients: up and down `(+0.1, 0)` and `(-0.1, 0)`, left and right `(0, +0.1)` and `(0, -0.1)`. It is added to the stepped position.

2. **Take the step back** (subtract the step vector times the speed factor). The nudge stays.
3. **Flush snap, along the facing only**, if the fraction of the coordinate is above zero (the fraction is 0 for a value just under a whole number): up `y = floor(y)`; left `x = floor(x + 0.5) - 0.5`; down `y = floor(y + 1)`; right `x = floor(x + 1.5) - 0.5`. The body's edge ends on a tile line. There is no wall test on the snapped position.

When the player is standing on a platform object or has a parent, the snaps are relative to the parent's position; without one the offset is 0.

After every step

Accepted or refused: `y` snaps to `floor(y + 0.0625)` if the fraction of y is under 0.0625 or over 0.9375. The same for x on `x + 0.5`: if its fraction is out of that band, `x = floor(x + 0.5 + 0.0625) - 0.5`. Then the position is stored. (A refused step still stores the position: that is how the nudge and the flush snap move the player.)

What this means for stopping positions

- Moving left or up, the probe is inside the body box; a body ends with its edge on the wall's tile line without any snap.
- Moving right or down, the probe is the tile line one tile beyond the body. The step that would put the body's right or bottom edge on a wall's tile line is refused (its probe reads the wall tile), and the flush snap then moves the body that half tile. Without the snap the body would stop half a tile short.
- A 2-wide body fits a 2-tile gap and not a 1-tile gap.
- Pressing against the end of a wall slides the body round it at 0.1 tile per tick: ten ticks to clear one tile.

Not read

See `open-questions.tsv`.

`client-internals/decomp-findings-2026-10-25-m/README.md`

Decomp findings 2026-10-25 (m): the whole walking rule

Derived facts from reading the walking routine of the 2.05, 2.17.1, 2.22 and 2.31 clients. It is the same routine in all four: the same probe table, nudge table, branches and constants. No addresses, no function names, no decompiler output, no code. Numbers are tile units.

- `01-walking-rule.md` — step order, the six probe points and how a point is turned into a tile, the corner nudge, the snap flush to a wall after a refused step, the 0.0625 line snap, what a point can be blocked by. Completes `../decomp-findings-2026-10-25-l/01-walking-body-box.md`, which had the probe table and the nudge conditions but not the snaps or the tile rounding.
- `open-questions.tsv` — what was not read.

Why this matters: a port that implements only "refuse the step if the first four points are blocked" stops half a tile short of any wall on its right or below (the right and bottom probes sit on the tile line one tile beyond the body) and sticks on corners. Seihai had exactly that; see its `documentation/walking-collision.md`.

Checked against: Seihai's tests of the rule (stopping positions from both half-tile alignments, the slide round a wall's end, the line snap). No lab run of the real client yet.

`client-internals/decomp-findings-2026-10-28-b/01-tile-type-tests.md`

Tile type tests (2.05 rev 2)

Date: 2026-10-28. From the client's per-type test, its wall test, the `arrays.dat` loader and the script expression handler (derived).

Which rule applies

Each level picks its tileset from the tile definitions (`addtiledef`). The type of the chosen definition decides how tile types are answered:

- **0, the default** (`pics1.png`): from the `arrays.dat` lists.
- **1 and 2, the new kind**: from the tile id alone, by a fixed rule.
- **5**: no board rule. Only NPC shapes have types.

An NPC shape that the player stands on (`setshape2` with tile types) answers first, for every tileset.

The default tileset

No 4096-entry type table is built. Each question asks one list, and looking up a tile is a binary search of a sorted list:

question	list
wall	not in array 0 (passable) and not in array 1 (liquid)
3 chair	array 4
4 bed top	array 5
5 bed bottom	array 6
11 water	array 1
12 lava	array 3
20 throw-through	array 2
21 jumping stone	array 8: four lists, one picked by the object's direction

Any other type number is answered only by NPC shapes, so a board tile is never type 0, 2 or 6 to 10 in this tileset.

Array 7 is not a type list. A tile's position in it (0–11, 12–19, 20–62, 63 and up) gives one of four ground classes, the same four that shape types 6 to 9 give. The horse uses the class to pick the sprite drawn under it.

Off the board, the tile reads as id 632. The wall test checks bounds first and calls off-board a wall. The type tests skip the bounds check and look up 632, which is in no list in the 2.22 file.

Consequences:

- **Lava passes the water test.** The liquid list holds lava too, so `onwater` and the swim test are both true on lava. A separate lava test counts ticks on lava. When the count passes 100, the client calls what looks like the hurt routine, and the count resets. Reading that call as the hurt routine is **inferred**.
- **The lava test in detail.** Once per player tick the client tests one point, the player's position plus (1.5, 2), against the lava list (array 3) only. On lava a counter goes up. On the 101st tick in a row it calls the hurt

routine with -0.5 (half a heart) and a flag that skips the usual "can be hurt" checks, then sets the counter to 0. A tick off lava sets it to 0. Which of the two offsets belongs to x was read from the order of two property reads, so (1.5, 2) rather than (2, 1.5) is **inferred**. The tick length was not read here.

- **A tile can be in several lists.** In the 2.22 file, 6 of the 133 throw-through tiles are passable, and 9 of the 76 lava tiles are not in the liquid list. All 9 are in the passable list, so they are walked on, not swum in, and the lava test still burns on them. (An earlier version of this note called them walls.)
- **Deep water is in no list.** A level made for testing (2026-10-28) uses a dark water tile for deep water. In the 2.22 file that tile is in none of arrays 0 to 3, so the wall test calls it a wall: deep water blocks. Shallow water in the same level is in the passable list only, so it is walked on.
- A thrown object is stopped by a tile that is not type 20 and that the wall test calls a wall.

Tilesets of the new kind

With `row = id / 16` and `col = id % 16`, a tile is a **wall** if any of these holds:

- id 32
- row 26 or 27 and col 2 or more
- row 30 to 47, 84 to 95, or 116 to 127
- row 128 or more and `(row / 16)` odd

A wall tile is 20 if its id is 512 to 767, 21 if it is in row 30 or 31 with col 8 or more, and 22 otherwise.

A tile that is not a wall is checked in this order, and the first match wins:

1. 288–319 gives 2
2. 320–383 gives 3
3. 384–399 gives 4
4. 400–415 gives 5
5. 64–287 gives 11
6. 192–287 gives 12. This step can never be reached, because step 5 has already taken those ids.
7. row 26 or 27 with col 0 or 1, or 768–799, gives 6
8. 800–831 gives 7
9. 832–911 gives 8
10. 912–943 gives 9
11. 944–991 gives 10
12. anything else gives 0

Script functions

- `tiletype(x, y)` returns -1 unless the level's tileset is of type 1, 2 or 5. For type 5 it always returns 1. Otherwise it applies the rule above (an NPC shape's type wins when it is above 1). On the default tileset, scripts therefore always get -1.
- `onwall(x, y)` is the wall test.
- `onwater(x, y)` is the type 11 test, true on lava as well.

The 2.00 to 2.04 builds all carry the three names. Their code was not compared.

client-internals/decomp-findings-2026-10-28-b/README.md

Decomp findings 2026-10-28 (b): tile types in 2.05

Derived facts from reading the 2.05 rev 2 client: how a tile's type is decided, and the script functions `tiletype`, `onwall` and `onwater`. No addresses, no function names, no decompiler output, no code.

- `01-tile-type-tests.md`: there is no type table for the default tileset. Each type test asks one `arrays.dat` list. Also covered: the fixed position rule for tilesets of the new kind, `tiletype()` returning -1 on the default tileset, and lava counting as water.
- `tile-type-lists.tsv`: which array or rule answers each type number.

Checked against: Seihai PR 108 (tests listed there). Static read only. Nothing was run against a server.

---

client-internals/decomp-findings-2026-10-28/01-sword-while-carrying.md

Sword key while carrying

Date: 2026-10-28. From the 2.05 client (derived). 2.22 and 2.31 not read.

The game keys reach the player by index: weapon (default D), sword (default S) and action (default A).

- **Sword key with an object carried:** no swing. The carried object is thrown, the same throw the action key does: it leaves from above the head, the carry sprite goes back to none (-1) and the idle gani is set. If the player's mode was walking, or the lay-down mode below, the mode goes back to idle. No swing precondition (sword power, swimming) is checked on this path.
- **Sword key with empty hands:** the normal swing.
- **Action key with an object carried:** throws, except in the lay-down mode, which has its own action.
- **Weapon key with an object carried:** does not throw. It only matters when the carried sprite is 239, 261 or 263; then the player enters a lay-down mode with its own gani. What that mode does was not read.

Online, a throw also triggers the client's player-props send (the carry sprite change); the exact bytes were not captured.

Not confirmed in a lab capture yet.

---

client-internals/decomp-findings-2026-10-28/02-item-list.md

Item list (Q key)

Date: 2026-10-28. From the 2.05 client (derived). 2.22 and 2.31 not read.

- The game actions by index: 0 up, 1 left, 2 down, 3 right, 4 item, 5 attack, 6 action, 7 map, 8 switchtochat, 9 itemlist, 10 pause. `itemlist` (default Q) opens and closes the item list. It opens from the normal state only when a player flag allows it (flag not identified).
- The list holds the player's weapons: built-in weapons and NPC weapons. A `/` in a weapon name makes a folder; folders of the current level come first, as 32 x 32 cells. Names starting with `-` are skipped when selecting.

- Layout: a framed window from (16, 80) to 16 px from the right and bottom edges; cells inside from (48, 112) to 48 px from the right and 64 px from the bottom. Cell size: max(image width, label width) + 10 by image height
    - label height + 15; cells are packed in the first free place, left to right then down. Image centred with 5 px above; label centred below; no-image weapons get a fixed 32 x 32 placeholder. Selected cell filled RGB(255, 212, 128). Font size 16. Scroll arrows when cells are cut off.
- Arrow keys: stretch the selected cell to the edge in that direction and take the overlapping cell with the closest near edge. Moving selects the weapon immediately; nothing is sent. Item key or Enter on a folder opens it; attack key goes up a folder. Other game keys close the list; Esc closes it. A mouse click on a cell selects it.
- Delete on the selected weapon removes it; online it sends PLI_NPCWEAPONDEL (29) with the weapon name, or PLI_NPCDEL (22) with the NPC id for an NPC that is not a weapon.

`client-internals/decomp-findings-2026-10-28/03-edge-links.md`

Edge links and the big map

Date: 2026-10-28. From the 2.05 client (derived). 2.22 and 2.31 not read. Not checked against a running client.

The level's neighbours

When a level is entered, the client fills a 3 x 3 table of level names around it from the big map: the text grid of quoted level names that the map screen uses (`maps/bigmap.txt` style, one row per line). It finds the current level's name in the grid and copies the eight names around it; cells off the grid stay empty. This happens only while a big map is loaded, and is skipped under one global flag (not identified; it may be the gmap or editor state). For each non-empty neighbour name the client also looks up a loaded level object of that name; that slot stays empty when the level is not loaded.

So the big map is not only a picture for the client: it decides which levels are "next to" the current one.

Following a link

Each tick the client takes a probe point: the player's position plus a per-facing offset. Then for each link of the level, in order:

1. **West edge link**: the link's x is 0, its width is 1, and its destination is the west neighbour's name (exact text compare). It fires when the probe x is 0 or less while facing left, or the player x is below -2.5, or, when the west neighbour level is **not** loaded, when the probe is inside the link's rectangle. Arrival: **x = player x + 64**, y unchanged. The link's own new-x/new-y are not used.
2. **East edge link**: x 63, width 1, destination = east neighbour. Fires when the probe x is 64 or more while facing right, or the player x is above 63.5, or (east neighbour not loaded) the probe is in the rectangle. Arrival: **x = player x - 64**, y unchanged.
3. **North edge link**: y 0, height 1, destination = north neighbour. Probe y 0 or less facing up, or player y below -3.0, or (not loaded) in the rectangle. Arrival: **y = player y + 64**, x unchanged.
4. **South edge link**: y 63, height 1, destination = south neighbour. Probe y 64 or more facing down, or player y above 63.0, or (not loaded) in the rectangle. Arrival: **y = player y - 64**, x unchanged.
5. **Any other link** (including an edge link whose destination is not the neighbour on that side): it fires when the probe is in the rectangle, except that a probe within 1 tile of an edge (x or y at most 1, or at least 63)

only counts while the player faces out across that edge. The arrival coordinates are the link's new-x and new-y, evaluated as expressions (`playerx` / `playery` give the player's current coordinate).

The first link that fires is taken; offline the client loads the level, online it asks the server.

So an edge link between two levels that sit side by side on the big map keeps the player's exact offset across the seam (east off x 63.6 arrives at x -0.4), whatever the link's new-x says. Without a big map, or for a level that is not the big-map neighbour, the link's own coordinates are used, and `playerx` on an east-edge link then arrives on the east edge of the destination.

Link records (for reference)

- A link line has the destination, x, y, width, height, and optionally new-x and new-y. A missing new-x or new-y, or one that is `-1`, means `playerx` / `playery`.
- The level editor's default for a new link on an edge: on the east edge (x 63) new-x is `0`; on the south edge (y 63) new-y is `0`; on the west edge (x 0, width 1) new-x is `61`; on the north edge (y 0, height 1) new-y is `61`. The other coordinate stays `playerx` / `playery`.

Open

- The probe offsets per facing were not read.
- The flag that turns the neighbour table off was not identified.
- Which big map a 2.05 client uses offline, and whether 2.22 does the same with the `PLO_BIGMAP` (171) map, is not read.
- How walking behaves past the edge when the neighbour level is loaded (the player x can go below 0 or above 63.5 before the link fires), and whether the neighbour's tiles are tested there, is not read.

`client-internals/des-and-hidden-strings.md`

Hidden strings and 2.22 saved passwords (2.05, 2.22)

Date: 2026-10-21

Tags: **2.05** read from the 2.05 rev 2 binary · **[2.22]** read from the 2.22 binary · **[cap]** checked in the lab (`tools/lab-setup.md`) · **[inf]** inference.

DES, saved passwords and the web cache (pointers)

The 2.05 DES routine (standard DES-ECB with each key byte bit-reversed), the 48-character string form and the 2.05 saved-password scheme are written up in `client-internals/saved-passwords.md`, and the web-level cache cipher in `formats/gwebl001-cache.md` (both added by PR #6). This note only adds the parts those files don't cover.

2.22 saved passwords [2.22] - Same 48-character scheme as 2.05, with one change. A saved value of 49 characters starting with a marker character is decrypted with a second key held by the options form. Any other value is decrypted with the first key, so old saved passwords still load. - New saves use the second key and the marker. - The account name is also stored hidden, using the byte transform below.

Hidden strings

2.05 listserver host [2.05][cap] - Uses the DES string form from `saved-passwords.md`. The server-list form builds an 8-byte DES key (`2B 77 77 A7 B4 31 21 41`) and a 48-character ciphertext one `Chr()` at a time, so neither appears in the string table. - It decrypts to `<list-host-classic>`. The port (`14921`) and the first packet text (`newmain`) are built the same way, as plain characters.

2.22 transform [2.22][cap] - `dec(b) = (NOT b − 10) & 0xFF` and `enc(b) = NOT (b + 10)`. - Used for: - the in-memory copy of the three listserver host names (`<list-host-reborn>`, `.<list-host-graalcc-suffix>`, `<list-host-graalbiz>`) and the port (`14922`) - the account name in the config - the account name before it goes into the game-server login packet - The host names and port themselves are read at start-up from `arrays2.dat`, where they are plain text (checked against `arrays2.sig`). Only the copy held in memory is transformed, so a memory search for the plain names finds nothing. See `protocol/listserver.md`.

2.22 version token [2.22] - The game-server login builds `GNW03014` one character at a time. - On the wire the login line is: `GCHAR 5` (client type 5, gen 5), `GCHAR key`, the token, `GCHAR len` + account, `GCHAR len` + password. **[cap]** - It is followed at once by id 37 (language) and id 39, a 10-byte value that is the same at every login of a given build (raw bytes, not two GINT5 words). What it is computed from is not traced. **[cap]**

Notes for tool writers

- To redirect 2.05, a hosts entry for `<list-host-classic>` is enough.
- To redirect 2.22, hosts entries for the three `listserver.*` names are enough. It tries them in order. Editing `arrays2.dat` instead would have to get past the `arrays2.sig` check, which the lab did not test.
- None of these schemes protect anything. They only keep the strings out of a casual `strings` dump.

`client-internals/drive-clients-inventory.md`

Drive clients set: what it contains (2026-10-24)

Derived facts from a shared Drive archive of old client builds (one zip, about 122 MB). It holds 73 exes in 55 folders. Full list with sizes and PE file versions: `version-profiles/drive-clients-exes.tsv`. The PE timestamps in these exes are all the same dummy 1992 date, so they say nothing about build date; only the file-version strings are useful, where present.

Versions present

- Early: 1st, 2nd, 3rd, 4th (+alpha), 5th (+alpha), 6th to 15th Graal ("Nth graal" folders), 1.1, 1.1.9 (+ Gserver), 1.21 (setup only), 1.28b2 (+ GServer, RemoteControl), 1.3b (GServer, RemoteControl), 1.31 to 1.39, 1.411, Graal 3D (+ GServer).
- 2.x: **2.00 (file version 2.0.0.0, 1,812,480 bytes) and rev1 (2.0.0.1), rev2 (2.0.0.2), rev3 (2.0.0.3)**; 2.01 (+ rev1); 2.02 (+ rev1, rev2); 2.03; 2.04; 2.05 (+ rev1, rev2).
- 2.1 test builds `graal21test2` and `graal21test3` (file version 2.1.0.0, about 1.75 MB). They still send the 2.05 token.
- NewMain 24 and 26 (file version 1.4.1.1, token `GNW08011`, with GraalShop, RemoteControl and LevelGen); NewWorld betas 1 and 2 (`Graal-NW.exe`, file versions 2.0.5.0 and 2.1.0.0).
- **Not present:** any 2.1.x release, 2.2x or 2.3x. There is no 2.17, 2.22 or 2.31 exe in this set, and nothing between 2.05 and 2.17 except the two 2.1 test builds. The 2.1 to 2.16 and 2.3 rows therefore stay inferred (`version-profiles/version-coverage-2026-10-24.md`).

- What the set does close: the missing real 2.00 exe, plus three revisions of it.

Running them (Wine)

- Wine 10 starts the 2.00, 2.05 rev2 and graal21test3 exes (the process stays alive and a window is created).
- Each one stops at "Cannot open file ... commands.rtf", because the Drive folders for 2.0x contain only the exe (and sometimes an `.idc`). The exes need their data files beside them: `commands.rtf`, the image folders and the language files.
- With the data files of another build (checked: 2.00 exe plus NewMain 26 data files) the 2.00 start screen draws normally.
- Run settings: new prefix, loopback-only lab network, private display. Nothing contacted a public network.
- Not run: any login to the lab, any 2.0x network flow, and the 1.x builds.

Candidates for the next decompile pass

2.01 to 2.04 and the two 2.1 test builds have no decompile yet. A static pass over them would settle the per-build handler differences in `version-profiles/plo-handled-ids-by-build.tsv`, which were read from dispatch tables only.

`client-internals/early-clients.md`

Early Graal clients — extracted details

Date: 2026-10-14

Derived from the decompile corpus. Raw exports stay in `/shdecompiling/`. This note is cleaned.

Lineage

Era	Binary	Port	Levels	Token
Zelda Online (Java, 1997–98)	applet	14801	`.zelda` (Z3-V1 RLE)	none
1st Graal (Delphi)	1st Graal.exe	14801	`.zelda` (24 refs) + `.graal` (4 refs)	none
1.39	Graal 139.exe	14900	transitional	`GNW07080`
2.05	Graal.exe v2.05	—	`.nw` (GLEVNW01)	`GNW03081`

1st Graal

- 505 functions. No version token. No `GNW` string.
- Default host: the loopback address, port `14801` (same as the Java precursor).
- Save format magic: `Graalsav1`.
- Level names hard-coded: `level1.zelda`, `cave2.zelda`, `house1.graal`, `village1.zelda`, and others. Both extensions coexist.
- Sign text present: `"Sign at ("`.
- String table includes `NZelda2#def`.
- **Room name is not always the constant `Zelda2`** (2026-10-24 decompile). One branch of the join routine uses `Zelda2`; the other, chosen by an argument given when the network client object is created, builds the room from the current level file's base name as `Z` + base + `2` (a level `level1.zelda` would give `Zlevel12`) and then runs a follow-up routine (not traced). Which branch normal play uses was not traced; the two-

client local mode is a likely user of the per-level form. The statement in `protocol/relay-1998.md` that 1st and 2nd Graal always join `Zelda2` holds for one branch only. The client always sends the name `def` in the join line and the real nickname later in a `D` payload.

1.39

- Token built as eight char immediates in the login/token builder: `G N W 0 7 0 8 0`.
- Connects to port `14900`.
- **Send helper drops payloads that contain a newline** (2026-10-24 decompile; the same helper exists in 2.05 rev2). A message is `id+32`, payload, newline, so a newline inside the payload would break the framing and the helper silently discards the whole message instead of sending it. The queue is flushed when it would pass 4095 bytes. In 1.39 the client closes the connection if more than about 256 KiB of compressed output is pending (the 2.05 figure was not checked). A client built on this framing should reject or escape payloads containing a newline rather than expect an error.
- Dhrake recovered ~390 names. Preferred corpus is `graal-139-dhrake/`.

Recovered classes

- `TAdventureClient_Create` / `InitTable`
- `TAdventureLevel_Create` / `InitTable`
- `TAdventureCommand_Create` / `InitTable`
- `TAdventureOperation_Create` / `InitTable`
- `TAdventureVariable_Create` / `InitTable`
- `TAdventureWeapon_InitTable`
- `TAdventureServer_InitTable`
- `TAdventureBoolOperation_InitTable`
- `TAdventure_DXDrawFinalize` / `DXDrawRestoreSurface` / `DXTimer`
- UI: `TAdventure_PlayButtonClick`, `SpeedButton`, `StatusBar`, `TabControl`, `HorzScrollBar`, `PaintBox`, `FormCanResize`

Script events present

`playerenters`, `playertouchsme`, `playerendsreading`, `playerlaysitem`, `playerisfemale`, `playerismale`, `playeronhorse`, `playerswimming`, `wasshot`, `shotbybaddy`, `shotbyplayer`, `compusdied`, `timeout`, `created`, `gotheart_`, `drawoverplayer`, `drawunderplayer`, `followsplayer`.

Properties

`hearts`, `rupees`, `shield`, `shieldpower`, `hasweapon`, `isweapon`, `weaponsenabled`, `playerx`, `playery`, `carriesnpc`.

Sounds

`arrow.wav`, `arrowon.wav`, `axe.wav`, `bomb.wav`, `chest.wav`, `compudead.wav`, `crush.wav`, `dead.wav`, `extra.wav`, `fire.wav`, `frog.wav`, `goal.wav`, `item.wav`, `item2.wav`, `jump.wav`, `nextpage.wav`, `put.wav`, `sign.wav`, `steps.wav`, `stonemove.wav`, `sword.wav`, `swordon.wav`, `thunder.wav`, `water.wav`.

Config keys (`graalconfig.txt`)

`serveraddr=`, `fixedport=`, `fixedudpport=`, `dontconnectlevels=`, `dontloadlistheads=`, `loadcolorsfromserver=`, `sortplayersbyname=`, `limitnicknames=`, `nicknamelimit=`, `gamemode=`, `language=`, `font=`, `fontsize=`, `musicon=`, `joystickon=`, `nodirectinput=`, `noudp=`, `nodecompressionerror=`, `nohtmlimages=`, `nohtmlinpms=`, `nomassmessages=`, `notoalls=`, `reversestereo=`, `saveimages=`, `compabilitydraw=`.

Files

`graal.bmp`, `graal.png`, `graalmap.bmp`, `levels\recordpos.txt`, `weblevels\`. Extensions: `.code`, `.gif`, `.graal`.

Rendering

DirectDraw and DirectSound. `TAdventure_DXDraw*` handles surface restore and the timer. `compabilitydraw=` is a config toggle.

2.05

- Token built as char immediates in `TAdventureClient_Create`: `GNW03081`.
- `DispatchGs1.txt` is the GS1 interpreter, not the packet dispatcher.
- Levels are GLEVNW01 `.nw`. Framing is length + zlib. Post-login uses the Borland LCG.

Protocol transition summary

1. Java precursor: text lines on 14801, `.zelda` RLE.
2. 1st Graal: same port, Delphi, `.zelda` dominant, no token, `Graalsav1` saves.
3. 1.39: port 14900, token `GNW07080`, full event set, DirectX, config file.
4. 2.05+: length + zlib, `.nw`, token `GNW03081`, post-login LCG.

Correction (2026-10-21, from `build-catalogue.md`): zlib packet compression starts with 1.2 (May 1999), and the post-login LCG insertion (gen 3) is already in 1.41 rev1 and every 2.0x build, not only 2.05+. 1.39 has no LCG insertion. `.nw` is in NewMain 1.42 and 2.05 but not in 2.00–2.04.

1998 betas to 1.32 (added 2026-10-21)

All 25 client builds from 1st Graal to 1.32 have now been analysed, together with the bundled GServer 1.1.9, 1.21, 1.28 and 1.3 and RemoteControl 1.21, 1.28 and 1.3. See:

- `protocol/relay-1998.md`: the gen-1 text relay (lobby lines, `D` payload types per build, game names `Zelda2` and `Graal` to `Graal8`).
- `protocol/gserv-1999.md`: the GSERV007 to GSERV018 login, zlib framing, which tokens each GServer accepts, and the id ranges.
- `formats/graalsav.md`: Graalsav1 to Graalsv11 and the DES-wrapped `.code` and `.gsave` files.

Two corrections to earlier assumptions:

- `TAdventureServer` (from 15th Graal) is a hostname-to-IP cache record, not a server or a peer-to-peer mode. The 1998 clients only ever connect out to a relay.

- Port 14801 is the compiled-in local default, and 14802 is the Cyberjoueur relay that the connect dialog points at. Neither one is a listening port in the client.

In the 1998 clients the network classes are `TAdventureClient` (the relay connection), `TOtherClient` (one per relay peer) and, from 3rd Graal, `TWebClient` (the HTTP level fetcher). The script classes listed above for 1.39 (`TAdventureLevel`, `TAdventureCommand` …) are not in 1st or 3rd Graal.

`client-internals/feature-specs/01-sound.md`

01 Sound and music

Snapshots read: Graal_analysis `8da6800` (main), Seihai `06aacfd` (main, 2026-10-24 03:23 JST). Tags: **2.05** read from the 2.05 client, **[srv]** GServer-v2 source, **[cap]** lab capture, **[inf]** inferred.

What it is

There is **no sound packet**. Sound is purely client-side and is triggered by four things: - GS1 commands `play`, `playlooped`, `stopsound`, `stopmidi`, `setmusicvolume`. - GANI `PLAYSOUND name x y` lines (the sound belongs to a frame). - Built-in events: item pickup, bomb, sword, arrow, baddy death, chest, jump, etc. - Config switches `musicon`, `soundfxon`, `reversestereo`, `usewinamp` (see 09).

The only network part is fetching a missing sound: the client sends `PLI_WANTFILE` (23) and the server answers with `PLO_FILE` (102) via `TPlayer::msgPLI_WANTFILE` / `sendFile`. File transfer is already in Seihai.

2.05 built-in effect names (the client plays these without any NPC): arrow, arrowon, axe, bomb, chest, compudead, crush, extra, fire, frog, horse2, item, item2, jump, lift, lift2, nextpage, put, steps, stonemove, sword, swordon, thunder, water (all `.wav`). **2.05**

Exact client behaviour (from `client-internals/sound-and-music.md`) 2.05

1. One entry point takes (file, level offset x/y, x, y). A leading `+` means loop and is stripped.
2. A missing file while online triggers WANTFILE; the sound plays on a later trigger.
3. Listener = player at (x+1.5, y+2). A sound from a neighbouring gmap level is shifted by 64 tiles. Pan = (soundx - listenerx) * 150, clamped to ±10000, negated by `reversestereo`. Volume = attenuation growing with distance on both axes, floored at -10000 (hundredths of a dB).
4. `.wav` goes to the effect path (only if `soundfxon`); anything else goes to the music path (only if `musicon`).
5. At most 8 effects play at once; a 9th is dropped. Effects are cached by file name. Leaving a level stops looped effects.
6. MIDI music loops until `stopmidi` or another track. `setmusicvolume l,r` is 0..1 each.
7. 2.17+ clients use BASS (effects, tracker modules, streams, URL radio). Same GS1 commands plus `musicpos`/`musiclen`. **[2.17][2.22]**

Seihai today

- `assets/gani.rs` parses `PLAYSOUND` into `GaniFrame.sound` (a name only). Nothing plays it.
- No audio dependency in `Cargo.toml` (only flate2, bzip2, minifb, eframe/egui).
- GS1 command table in `scripting/gs1.rs` lists the sound commands by opcode but there is no executor for them.

Steps for Claude

1. Add a `sound` module with a trait `SoundOut { play(name, pan, vol, looped), stop(name), stop_all_loops() }` and a null implementation, so the rest compiles and tests without a device.
2. Write a small WAV reader (PCM 8/16-bit, mono/stereo; the 2.05 files are plain PCM). Cache decoded buffers by lower-cased base name.
3. Implement the mixing rules above: 8-voice cap, pan and volume from listener distance, + loop prefix, `reversestereo` flag.
4. Real backend behind a cargo feature (for example `rodio` or `cpal`) so the default build stays dependency-light. Ask the user before adding the crate.
5. Wire three triggers: GANI frame `sound` when a frame is first shown, the GS1 `play` family in `offline/mod.rs` and `scripting`, and a short fixed list of built-in events (sword swing, bomb, item pickup, chest).
6. Missing file: call the existing `send_wantfile`, then retry on the next trigger.
7. Music: play only formats you can decode (WAV, later OGG/MP3). Do not try to emulate MIDI; log "music skipped: midi" and carry on. This is a design call for the user.

How to test

- Unit tests with a recording `SoundOut` mock: feed a gani with `PLAYSOUND`, check calls, the 8-voice cap, pan sign and the loop prefix.
- Lab: the lab gserver world has a `sounds` folder; log in with Seihai, walk, and request a sound file with WANTFILE to see the `PLO_FILE` path (private display only).

NEEDS

nothing for `.wav` effects. (MIDI/MP3 backend choice is a design decision, not missing research. The exact 2.22 extension-to-backend dispatch is only needed if you want BASS parity.)

`client-internals/feature-specs/02-weapons-combat.md`

02 Weapons and combat

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 (local checkout).

Weapon lists (server to client) srv

PLO	Name	Layout
33	NPCWEAPONADD	`str name` + NPC prop stream (image, script). A name the player already has is ignored. 2.05 fires `playerenters` on it.
34	NPCWEAPONDEL	rest = name
43	DEFAULTWEAPON	`c item index` (0-24, see 03). A built-in weapon: bow, bomb, sword, etc.
197	(bytecode)	v4+ only; replaces the script for GS2. 2.05 never sees it.

Client to server: `PLI_WEAPONADD` (33) type 0 = `c item index`; type 1 = `i3 npc id`. 2.05 only sends type 0. `PLI_NPCWEAPONDEL` (29) = name.

GServer builds the list at login (`sendLogin`): clear (`PLO_CLEARWEAPONS`), the account's weapons, protected weapons, then default weapons if the `defaultweapons` option is on. The weapon script text comes from `TWeapon`; for pre-v4 clients the script is sent as plain GS1 text.

Combat packets srv[cap]

The client is authoritative. It sends its own action and the server re-emits it to the rest of the level with `s player id` in front. In 2.05 most of these also go over UDP (see 07); the TCP copy is sent only when some player in the level has no UDP port, or an NPC server exists.

PLI	PLO	Name	Client layout	Notes
4	11	BOMBADD	`c x*2, c y*2, c power(+8 = mine), c fuse`	fuse seen: 55 (about 2.75 s)
5	12	BOMBDEL	`c x*2, c y*2`	PLO has no player id
9	19	ARROWADD	`c x*2, c y*2, c flags, c type`, last byte offset 33	flags: dir&3, +4 effect, +8 owned
10	20	FIRESPY	`c (a*8+b)`	
11	21	THROWCARRIED	empty	
26	40	HURTPLAYER	`s victim, c (dx+4)*16, c (dy+4)*16, c power` [+ `i3 npc`]	receiver: power = (v-32)/2 hearts, cap 1.0
27	36	EXPLOSION	`c radius, c x*2, c y*2, c power(+8 mine)`	power & 7 clamped 1-3
36	46	HITOBJECTS	`c power*2, c x*2, c y*2` [+ `i3 npc`]	sword-swing "hit" broadcast
16	27	BADDYHURT	`c id, c dx, c dy, c power`	applied only by the level leader
15	2	BADDYPROPS	`c id` + baddy prop stream	server forwards to everyone except the leader
17	-	BADDYADD	`c x*2, c y*2, c type, c power` + image	server creates the baddy and answers with props
38	48	TRIGGERACTION	`i3 npc, c x*2, c y*2` + action text	half-tiles in 2.05

Server behaviour (`TPlayer.cpp`): - `msgPLI_HURTPLAYER`: victim lookup; if the victim has the paused status bit, nothing happens; otherwise forwards to the victim with the attacker id. - `msgPLI_BADDYHURT`: sent only to the level leader (first player in the level's list), who owns baddy AI. `msgPLI_BADDYPROPS` stores the props and forwards to non-leaders. `msgPLI_BADDYADD` caps power at 12 half-hearts and marks it non-respawning. - `TLevelBaddy::setProps`: mode die starts a 2 s timer to dead; mode dead starts the `baddyrespawntime` timer (60 s lab default) or removes the baddy; with `baddyitems` it drops an item on death. - Level objects (`TLevel::doTimedEvents`) respawn cut bushes after `respawntime` (15 s) by re-sending BOARDMODIFY. - `msgPLI_CLAIMPKER` (14): the dead player names the killer; the server adjusts kills/AP (and a rating update in sparring zones). - `PLO_ISLEADER` (10): "you run this level's baddies". Seihai counts it as unhandled. It matters only if Seihai wants to simulate baddies. - Spin attack (2.1+ only, `PLPROP_GANI` = `spin`, handled in `TPlayerProps.cpp`): the server itself broadcasts four `HITOBJECTS` packets around the player (4 half-tiles above, below, left, right of the centre) so other clients register the spin. 2.05 has its own sender for HITOBJECTS (36); the exact trigger in a normal swing is not traced. How 2.05 receives hits is in `protocol/packet-layouts.md`

"Hits": HURTPLAYER applies only if the attacker is within 10 tiles and the local player is hurtable; HITOBJECTS only if the player's centre is within 2 tiles of (x, y).

2.05 sword swing and hit box [2.05][dec]

From the 2.05 rev2 decompile (2026-10-24); details in `../decomp-findings-2026-10-24/04-sword-hitbox.md`. - **Start of a swing.** Needs weapons allowed by the level or server, sword power not 0, not swimming without a horse, and not already in the charged-spin mode. A normal swing is mode 6. A spin is mode 13: it needs the has-spin flag (STATUS bit 0x40) and starts when S is pressed again while mode 6 is still running. It is not a hold-to-charge; see `../player-mannerisms-live.md`. The client plays the sword, ridesword or spin gani and a sound, then checks for a cuttable bush at the sword tip. The sprite sent to others follows animation steps 9 to 13 (S 6 to 10, see 11). - **The victim's client decides a hit**; the attacker only broadcasts position, direction and sprite. A hit counts when **all** of these hold: - the attacker is on step 9 to 13 (S 6 to 10); - the victim is **0 to 4 tiles ahead** of the attacker in the facing direction; - the sideways offset is **under 2 tiles**; - the victim's **AP is not exactly 100** (such players cannot be hit); - the victim is hurtable (not in mode 3, mode 10, dead or editor/pause). - **Damage** is inferred to be sword power divided by 2 hearts. Not confirmed from a capture; treat it as a Seihai default. - **Explosions:** a bomb hit needs the victim within 1 tile on both axes of the blast, with knockback of 0.5 or 1.0 tile. - **After a hit** the victim goes to hurt mode 3 for 40 ticks (cannot be hurt again), and if hearts reach 0 or below the victim client sends `PLI_CLAIMPKER` (14) naming the attacker (see 04). - Open: how this relates to the `HURTPLAYER` (26) packet in 2.05 (the decompile findings do not say whether the attacker also sends it for a sword hit); frame timing in milliseconds (comes from the gani files, not the client); 1.39 sword and hit box (not read).

Seihai today

- `SessionState` keeps `weapons` (names and scripts) and `default_weapons`; it does **not** show or select them, send any combat packet, or decode 11/12/19/20/21/36/40/46/48 (they count as unhandled).
- Offline mode has a sword model in the web-Seihai design notes (`formats/seihai-systems.md`). Those numbers are Seihai's own, not 2.05 values.
- No HURTPLAYER handling: Seihai never loses hearts from another player.

Steps for Claude

1. Parse server packets 11, 12, 19, 20, 21, 36, 40, 46, 48 into events on `SessionState` (list of timed objects: bombs, arrows, hits). Decode per the table. Ignore silently if the level is not the named level.
2. Render bombs and arrows with the existing sprite sheet; remove on BOMBDEL, on explosion or timeout.
3. HURTPLAYER receive: subtract power from `CURPOWER` locally, apply knockback `(v-0x60)/16`, start an invulnerability window, then send the new `PLPROP_CURPOWER` (the server accepts hearts from the client, and refuses to raise them for AP under 40). 3a. Seihai as a **victim** should evaluate sword hits itself with the 2.05 rule above (reach 0 to 4 tiles ahead, lateral under 2, attacker on S 6 to 10, own AP not 100, not already hurt or swimming), and send CLAIMPKER when hearts reach zero.
4. Send side, in this order: BOMBADD (4) with the owner flag, explosion (27) when the fuse ends, BOARDMODIFY (1) for any tile the blast changes, HITOBJECTS (36) when your sword or bomb effect lands (trigger not traced, see NEEDS), HURTPLAYER (26) when your sword box hits another player in the level.
5. Weapons UI: list `weapons` and default weapons; pick with the item key; for GS1 weapons run the script through the existing `scripting` module.
6. Baddies: parse BADDYPROPS (2); only simulate if `ISLEADER` (10) has been received. First pass: draw them from props only.

7. Track an invalid-packet budget: GServer disconnects a client after the 6th unknown id (`msgPLI_NULL`), so never send ids absent from the profile's `pli` set.

How to test

- Two Seihai or Seihai plus one Wine 2.05 client in the lab level, `bushitems = true`. Drop a bomb next to bushes: expect BOMBADD, then eight BOARDMODIFY runs, ITEMADD drops, then a respawn BOARDMODIFY 15 s later (matches the existing capture in `packet-layouts.md`).
- Replay the existing decoded two-player captures through `SessionState` (examples `lab_replay`) and assert the new event lists.
- GServer rebroadcast can be checked with a second raw client from `examples/session_probe.rs`.

NEEDS

Nothing for the protocol. The 2.05 hit-box geometry and the frames that can hit are now known (above). Still open, for feel only: when exactly 2.05 sends `HITOBJECTS` (36) in a normal swing, the real damage formula (inferred power / 2), and the 1.39 sword. The server does not check any of it, so any plausible box interoperates.

`client-internals/feature-specs/03-inventory.md`

03 Inventory

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 (`TLevelItem.cpp`, `TPlayerProps.cpp`, `TPlayer.cpp`).

What it is

The player's counts and gear are **player props sent by the client**. The server clips and stores them. Items on the floor are `PLO_ITEMADD/ITEMDEL`; chests are `PLO_LEVELCHEST`.

Item index table (0-24) [srv], used by ITEMADD, ITEMTAKE, chests, DEFAULTWEAPON, WEAPONADD

0 greenrupee (+1 gralat), 1 bluerupee (+5), 2 redrupee (+30), 3 bombs (+5, max 99), 4 darts (+5 arrows, max 99), 5 heart (+1 heart), 6 glove1 (glove power at least 2), 7 bow, 8 bomb, 9 shield (power 1), 10 sword (power at least 1), 11 fullheart (+1 max hearts, hard cap 20, and heals fully), 12 superbomb, 13 battleaxe (sword at least 2), 14 goldensword (sword 4), 15 mirrorshield (2), 16 glove2 (3), 17 lizardshield (3), 18 lizardsword (sword at least 3), 19 goldrupee (+100), 20 fireball, 21 fireblast, 22 nukeshot, 23 joltbomb, 24 spinattack (sets the has-spin status bit). Items 7-8 and 12 and 20-23 become weapons (`addWeapon`). Shields and swords never downgrade. Rupees cap at 9,999,999.

Props involved (self stream, PLPROP ids) [srv]

1 MAXPOWER (hearts), 2 CURPOWER (half-hearts), 3 RUPEESCOUNT (i3), 4 ARROWSCOUNT, 5 BOMBSCOUNT, 6 GLOVEPOWER, 7 BOMBPOWER, 8 SWORDPOWER (`c(power+30)` + image, or a plain power 0-4 for `swordN.png`), 9 SHIELDPOWER (`c(power+10)` + image, or plain 0-3), 18 STATUS bits (`0x40` has spin). Server clipping when the client sets them (`setProps`): arrows and bombs 0..99; rupees at most 9,999,999; sword and shield power clipped to `swordlimit`/`shieldlimit` (3 in the lab); bomb power 0..3; glove max 3; **CURPOWER cannot rise for players under 40 AP**. With V8NPCSERVER some of these become server-only; the lab server is built without it.

Item pickup and chests [srv][cap]

- Floor item pickup: the client sends `PLI_ITEMDEL` (13) `c x*2, c y*2`. The server relays `PLO_ITEMDEL` (23) to everyone else and removes it from the level. It does **not** give the item on 13. `PLI_ITEMTAKE` (32) does the same removal and also applies the item to the player's props (via the table above) and echoes the prop change to the player and the level. 2.05 used 13 in the capture (client then edits its own props). **[cap]**
- Dropping an item: `PLI_ITEMADD` (12) `c x*2, c y*2, c item`; the server relays `PLO_ITEMADD` (22), adding a player id for dropped swords and shields.
- Chest open: `PLI_OPENCHEST` (20) `c x, c y` (whole tiles). If the account has not opened it (`chestList` in the save), the server applies the chest item with the same table, answers `PLO_LEVELCHEST` (4) `c 1, c x, c y`, and records the chest. Level entry sends one LEVELCHEST per chest (`c opened, c x, c y` + `c item, c signindex` if closed). The account's opened-chest list persists (see 05).
- Drops: bushes/vases with `bushitems`, baddies with `baddyitems`, and deaths (`dropitemsdead`, `mindeathgralats`..`maxdeathgralats`, `dropItemsOnDeath`) create ITEMADD packets from the server.
- Weapon list display is in 02.

Seihai today

- `send_open_chest` exists; `main.rs` sends it on A/Space near a chest and takes the reply (`opened_chests`) into the session. The item itself is not applied: the chest item index is printed to stderr only.
- HUD draws `player_health` and `rupees` (`render/tiles.rs draw_hud`). Bombs, arrows, sword, shield, glove, MAXPOWER, AP, status are not displayed or sent.
- ITEMADD/ITEMDEL (22, 23) are unhandled. Own-prop sending is limited to position/sprite/chat.
- Offline mode has its own item model (`offline/mod.rs give_item`) that is not the GServer table.

Steps for Claude

1. Add an `Inventory` struct to `SessionState` filled from the self prop stream: maxpower, curpower, rupees, arrows, bombs, glove, bombpower, sword (power+image), shield (power+image), status bits. Fix decode for both prop forms (see `protocol/props.rs`).
2. Parse PLO 22/23 into a per-level list of floor items (x,y,item); parse PLO 4 chests; draw both.
3. Pickup: when the player's feet overlap a floor item, send `PLI_ITEMTAKE` (32) if the version profile has it (2.05 lists 32), and let the server echo the new props; fall back to 13 plus local prop edits if a server ignores 32.
4. Chest: after the server's LEVELCHEST (opened=1), show the item name from the table and update the inventory locally only if no echo arrives within one tick (the lab echoes via `setProps`).
5. HUD: hearts (CURPOWER/2 of MAXPOWER), rupees, bombs, arrows, selected weapon.
6. Implement an item key (`Q` in the original) to select among default weapons and named weapons; sending a use event is 02's job.
7. Add a unit test per table row (item index to prop change) and a replay test using the lab chest/bush captures.

How to test

- Lab, one Seihai: log in, walk to a bush, bomb it, take the drop, and verify the prop echo. Chest in `onlinestartlocal.nw` if present; otherwise use the RC-free approach of adding a chest to the lab level file (Seihai supports `.nw` reading, the editor in `tools/level-editor` can place one).
- Check server saved state afterwards in the lab account file (`servers/lab/accounts/<name>.txt`): RUPEES, BOMBS, ARROWS, SWORDP, CHEST lines.

NEEDS

nothing blocking. Optional: "the 2.05 item/weapon list window (Q key) layout" for a faithful HUD; not needed for interoperability.

`client-internals/feature-specs/04-death-respawn.md`

04 Death, respawn and health regeneration

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 `TPlayerProps.cpp`, `TPlayer.cpp`, `TAccount.cpp`.

What it is

Death is **client-driven**. When hearts reach zero the client sets the DEAD bit in its STATUS prop (bit `0x08` of prop 18), sends the prop, and (if a killer is known) sends `PLI_CLAIMPKER` (14) `s killer id`. The server reacts to the bit transitions; it does not move the player or heal gradually.

Exact server behaviour (`TPlayerProps.cpp`, STATUS case) [srv]

- **Dies** (bit goes 0 to 1): unless the level is a sparring zone, `deaths` increases and `dropItemsOnDeath` runs (random gralats between `mindeathgralats` and `maxdeathgralats` as a floor item, plus items if `dropitemsdead`). If the dead player was level leader and others are present, leadership passes to the next player and that player gets `PLO_ISLEADER` (10).
- **Revives** (bit goes 1 to 0): the server sets hearts by AP: AP under 20 gives 3 hearts, under 40 gives 5, otherwise the full MAXPOWER; at least 0.5. It pushes `PLPROP_CURPOWER` to the player and the level, and re-sends `ISLEADER` if the player is leader. **There is no warp**: where the player respawns is a client decision.
- AP (the "alignment" prop 32): with `apsystem` on, `TPlayer::doTimedEvents` counts `apCounter` down once a second (not while paused or in a sparring zone). At zero AP goes up by 1 (max 100), the server pushes `PLPROP_ALIGNMENT` to the player and the level, and reloads the counter from `aptime0..4` (30/90/300/600/1200 s in the lab, by AP band 0-19, 20-39, 40-59, 60-79, 80+). Entering a sparring zone at 100 AP drops it to 99.
- `PLI_CLAIMPKER` (`msgPLI_CLAIMPKER`): ignored for self-kills or an unknown killer. In a sparring zone a Glicko-style rating update runs for both players (skipped if both share an IP) and `lastspartime` is set. Elsewhere the killer gets +1 kills (unless `dontchangekills`), and if `apsystem` is on and the victim has 20+ AP and the killer has any AP, the killer loses AP by a formula based on both AP bands and the killer's counter is reloaded.
- Heart regeneration is not periodic on the server. Hearts come from heart items (03), `healswords`, the revive rule above, or script/NPC props. A client-sent CURPOWER above the server value is refused while AP is under 40.
- Login saves/loads HP (`HP`, `MAXHP`, `AP`, `APCOUNTER`, `KILLS`, `DEATHS`) in the account file (05).
- Idle: `disconnectifnotmoved` + `maxnomovement` (1200 s in the lab) kicks idle players with a disconnect message (see 13).
- Baddy death timers are in 02 (2 s to dead, `baddyrespawntime` 60 s).

What 2.05 does on the client side [2.05][dec]

From the 2.05 rev2 decompile (2026-10-24). Tick = 50 ms. Behaviour only; tables are in `../decomp-findings-2026-10-24/` (`01-death-respawn.md`, `constants-205.tsv`, `life-states.tsv`, `player-modes.tsv`).

Trigger. Any change that leaves hearts at 0 or below starts the death sequence once. A per-frame safety net also forces the dead life-state whenever hearts are 0 or below.

Sequence on the dying client 1. A death timer starts at 0 and counts up each tick; it **stops at 30 (1.5 s)**. 2. Life-state becomes dead, mode returns to 0 (normal) and the player faces down. 3. The local `dead` gani plays and the script event `playerdies` fires. 4. **Controls are ignored until the timer passes 27 (1.35 s)**, so input works from timer 28. 5. The STATUS prop (18) gets the dead bit (0x08) **only when life-state is dead and the timer is at least 28** (about 1.4 s after death). Other players therefore see the death about 1.4 s late. 6. No dead sprite goes on the wire: prop 17 shows idle plus direction (see 11). The dead state is carried by the status bit only.

Respawn - After the timer passes 27, any movement or action key respawns the player. **There is no respawn packet and no server permission step** on this path. After respawn the client clears the status bit 0x08 when life-state returns to playing, and the server's revive rule (above) sets hearts and pushes CURPOWER. - Hearts the client sets, by AP: below 20 gives 3; 20 to 39 gives min(5, max hearts); 40 or more gives max hearts. (Same bands as the server's revive rule.) - **Destination** = the last saved level-entry point (level, x, y, direction). It is updated on every level entry (unless the old and new levels share the same "name group", see open items) and at login. If nothing was saved and the client is online: x 30.5, y 30.0, facing down, in the current level. Offline: the default start level `tutorial_b-4.graal`. - When the online flag is set the client **re-sends the current level at the current x/y**, so the server sees an ordinary level warp (`PLI_LEVELWARP` route), not a "died" message.

CLAIMPKER. The *victim's* client sends `PLI_CLAIMPKER` (14) with the attacker's player id when a sword hit, an explosion hit or a UDP hurt leaves hearts at 0 or below. The server's reaction is in `TPlayerProps.cpp`/`TPlayer.cpp` (above); it is not in the client binary.

Damage rules that feed this - Healing only happens at AP 40 or above. - No damage in swim mode (10) or in the editor/pause state unless forced. Hurt state: mode 3 for 40 ticks, `hurt` gani, knockback, script event `playerhurt`. A player already in mode 3, mode 10 or dead cannot be hurt. - Hearts, AP and max-power sit in an attribute array the client stores obfuscated twice and cross-checks; a mismatch raises "memory problems error 1/2". Editing hearts from outside the client is detected by the client itself. A Seihai that holds hearts in plain fields has no such check (fine, the server stays authoritative).

Open - The exact "same name group" rule for *not* updating the saved entry point was only partly traced. Until it is known, update the saved point on every level entry and treat a warp inside one map or group as a Seihai choice. - The server-side reaction to CLAIMPKER is described above from GServer-v2, not from the client. - The `0x20` STATUS bit is the `hidesword` timer (`../status-bit-0x20.md`); a few mode values are unlabelled (see 11). - ~~1.39 death handling~~ — filled in `../../version-profiles/matrix/rules-139.md` (same 30-tick timer and STATUS bit rule as 2.05; hearts to max, no AP bands). 1st Graal still open.

Seihai today

- Reads `player_health` for the HUD. Does not send STATUS, does not track the dead bit, does not send CLAIMPKER, and treats `ISLEADER` (10) as unhandled.
- Offline: the web-Seihai design (`formats/seihai-systems.md`) has its own death panel and revive (85% of rupees kept, spawn at level start). Seihai's own invention, fine for offline.

Steps for Claude

1. Track CURPOWER from the server. Add `dead: bool` and a death tick counter (50 ms ticks, capped at 30). When hearts become 0 or below after a hit (02), start the timer, play `dead`, and send CLAIMPKER (14) when the hit has an attacker id. Set the DEAD bit (0x08) in STATUS and send it only once the timer reaches 28 (2.05 behaviour).
2. While dead: show the `dead` gani frame (or a placeholder), block movement and weapons until the timer passes 27. Send prop 17 as idle plus direction.
3. Respawn on any movement or action key once the timer has passed 27 (the 2.05 rule, no timer-only respawn): clear the bit, send STATUS, accept the server's CURPOWER push (hearts by AP, as in the table above). Respawn location: the last saved level-entry point (level, x, y, direction), updated on every level entry; if none, x 30.5, y 30.0 facing down in the current level. Re-send the current level at that x/y as an ordinary level warp. No respawn packet exists. Keep the saved point configurable only for the open "same name group" case.
4. Show gralat loss from the floor item the server drops (ITEMADD from 03).
5. Handle `PLO_ISLEADER` (10) by storing `is_leader` (needed for 02 baddies).
6. Show an HP regen hint only if the server pushes CURPOWER; do not simulate regen.

How to test

- Lab: Seihai sends the dead bit (STATUS prop with bit 0x08) and then clears it. Check the lab account file: `DEATHS` increments, and the server pushes CURPOWER = full hearts (the default account has AP 50, so the rule gives MAXPOWER, 3 hearts). Set AP under 40 in a copy of the account to see the 5-heart and 3-heart bands.
- Second client observes `PLO_OTHERPLPROPS` with the STATUS dead bit.

NEEDS

Filled by the 2026-10-24 2.05 decompile: timings, input lock, status-bit delay, hearts by AP, destination rule, and that no respawn packet exists. Server side was already known.

Still open: the "same name group" rule for the saved respawn point, a few unlabelled mode values, and the 1st Graal death handler (1.39 is in `../../version-profiles/matrix/rules-139.md`). A lab capture of a real 2.05 client dying twice would confirm the sequence on the wire (status bit timing, the warp frame after respawn).

`client-internals/feature-specs/05-save-load.md`

05 Player save and load

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 `TAccount.cpp`, `TPlayer.cpp` (`cleanup`), `TPlayerLogin.cpp`.

What it is

For every client from 2.00 on, **the server owns the save**. The 2.05 client contains the string "Save games are not supported anymore", and the offline F2/F3 save games ended with 1.41. The client has nothing to save except its local files (`graalconfig.txt`, `ignoredaccounts.txt`, `buddyaccounts.txt`, per-account profile file, web-level cache `GWEBL001`).

What a client must do is **consume the login burst correctly**, because that burst is the loaded save.

Server file (GRACC001, `accounts/<name>.txt`) [srv]

Text file, first line `GRACC001`, then `KEY value` lines, CRLF on write. Keys read by `TAccount::loadAccount`: NAME, NICK (max 223 chars), COMMUNITYNAME, LEVEL, X, Y, Z, MAXHP, HP, RUPEES, ANI, ARROWS, BOMBS, GLOVEP, SHIELDP, SWORDP, BOWP, BOW, HEAD, BODY, SWORD, SHIELD, COLORS (5 comma-separated), SPRITE, STATUS, MP, AP, APCOUNTER, ONSECS, IP, LANGUAGE, KILLS, DEATHS, RATING, DEVIATION, LASTSPARTIME, FLAG (`name=value`, repeated), ATTR1..ATTR30, WEAPON (repeated), CHEST (repeated: `x:y:level` style string), then the admin block: BANNED, BANREASON, BANLENGTH, COMMENTS, EMAIL, LOCALRIGHTS, IPRANGE, LOADONLY, FOLDERRIGHT (repeated), LASTFOLDER. - Unknown account: the server clones `accounts/defaultaccount.txt`, applies `startlevel`/`startx`/`starty` from `serveroptions` if set, and saves the new file. - `guest` accounts are load-only and get a random `pc:` community name. - Saved on disconnect (`TPlayer::cleanup`, only clients that are not load-only) and every 300 s while connected (`doTimedEvents`). Not saved if the listserver login failed.

How the save reaches the client (login burst, `sendLogin`) [srv][cap]

Order (`TPlayer::sendLogin` then `sendLoginClient`): signature, own props (`PLO_PLAYERPROPS` 9 with the loaded values; no level or position props), `PLO_CLEARWEAPONS`, one `PLO_FLAGSET` (28) per saved player flag, the server flags, `NPCWEAPONDEL` for "Bomb" and "Bow" (the client adds them itself), the account's weapons (`NPCWEAPONADD` 33 and `DEFAULTWEAPON` 43), protected weapons and classes, then the warp: `PLO_PLAYERWARP` (14) to the saved level and x/y followed by the level data. After the level come bigmap/minimap/RPG window (2.1+), `PLO_STARTMESSAGE` (41) and an empty `PLO_SERVERTEXT` (82). Chest open state arrives inside each level's chest packets (03). The saved `LEVEL`, `X`, `Y` therefore arrive as the first warp, not as props. Level and position are stored, so reconnecting resumes where the player left.

Seihai today

- Consumes the burst: props (partly), flags (`SessionState.flags`), weapons, default weapons, warp, board, chests list from LEVELCHEST (`opened_chests`).
- No local save. Offline mode in web-Seihai design notes has its own JSON save (`SaveV1` under `formats/seihai-systems.md`), not related to GRACC001.
- Seihai writes only its own `settings.rs` file.

Steps for Claude

1. Nothing to implement for online play beyond keeping the full self prop stream (03) and flags (done).
2. Keep `login_state` in the session: level, x, y from the first `PLAYERWARP`; log it in `session_probe` output (names only).
3. Optional offline save: keep the existing offline model; do not invent a GRACC001 writer in the client. If a local server is wanted, use GServer-v2 in the lab instead.
4. Add a lab test that logs in twice and asserts the second `PLAYERWARP` equals the position saved at the first logout.

How to test

- Lab: log in with the placeholder account, move, send a chat, disconnect, then read `servers/lab/accounts/<name>.txt` (X, Y, LEVEL, HP, RUPEES). Reconnect and confirm the warp matches. Never print credentials.
- Or replay a captured login burst through `SessionState`.

NEEDS

nothing.

`client-internals/feature-specs/06-gmap-overworld.md`

06 GMAP overworld and map links

Snapshots read: Graal_analysis `8da6800` (`formats/gmap.md`), Seihai `06aacfd`, GServer-v2 (`TMap.cpp`, `TPlayer.cpp` `setLevel`/`sendLevel`/`msgPLI_ADJACENTLEVEL`, `TPlayerProps.cpp`, `TLevelLink.cpp`, `TPlayerLogin.cpp`). Folded here from the public-server report: **PLO_BIGMAP 171, PLO_MINIMAP 172, PLO_SETACTIVELEVEL 156, PLO_LEVELMODTIME 39**.

Three different "maps" (do not mix)

1. **Level links** (`PLO_LEVELLINK` 1): `dest x y w h newx newy` text per link. Walking into the rectangle makes the client send `PLI_LEVELWARP` (0) / `PLI_LEVELWARPMOD` (30). `newx`/`newy` may be `playerx`/`playery`. `TLevelLink::parseLinkStr` accepts level names with spaces. Seihai parses PLO 1 and has `send_levelwarp`.
2. **bigmap-style server maps** (`maps =` option, `bigmap.txt` text grid, `MapType::BIGMAP`): server only. It lets the server know which levels are adjacent so players are visible and can chat across edges. The client does nothing special. Level-to-cell data comes from `TMap::loadMapLevels`.
3. **GMAP** (`gmaps =` option, `GRMAP001` file, `MapType::GMAP`): the client walks continuously across level borders. Separate again: `bigmap =` / `minimap =` options (4 comma fields `text,image,x,y`) make the server send `PLO_BIGMAP` (171) and `PLO_MINIMAP` (172) at login (2.1+, in `sendLoginClient`). These are overlay settings for the client's map screen and minimap: file names to fetch plus a default x,y for heads of players off the map. Harmless to skip. The notes warn that setting `bigmap` breaks gmaps.

GMAP wire behaviour [srv]

- File keys (`formats/gmap.md`): WIDTH, HEIGHT, LEVELNAMES block (quoted CSV rows), MAPIMG, MINIMAPIMG, NOAUTOMAPPING, LOADFULLMAP, LOADATSTART.
- Entering a gmap level (2.1+ clients): `PLO_PLAYERWARP2` (49): `c x*2, c y*2, c z+50, c gmapX, c gmapY` + gmap file name. 2.05 has no handler for 49 and gets plain `PLAYERWARP` (14).
- In `sendLevel`: after the level's board, links, chests, horses, baddies, the server sends `PLO_LEVELNAME` (6) with the **gmap name** (not the level) and, when this is the first level, `PLO_SETACTIVELEVEL` (156) with the gmap name before the NPC props, then `PLO_GHOSTICON` 0.
- Neighbours: the client sends `PLI_ADJACENTLEVEL` (35) `i5 modtime + level`; the server replies with that level's data (`sendLevel(..., fromAdjacent)`), then restores `LEVELNAME` to the gmap. The 3x3 neighbourhood is preloaded this way.

- Crossing a border is **client-driven**: the client updates `PLPROP_GMAPLEVELX` (43) / `GMAPLEVELY` (44) and X/Y in gmap coordinates. For each of those props the server looks up the new level name in the map (`TMap::getLevelAt`), leaves the old level and enters the new one with an unknown mod time.
- Props: `PLPROP_CURLEVEL` carries the gmap name while on a gmap; X/Y inside a gmap level are level-local plus the cell; newer clients (2.30+) also send X2/Y2/Z2 pixel props (78-80).
- UDP id 24 ("entered level") goes to peers in the 3x3 neighbourhood (see 07).
- `PLO_LEVELMODTIME` (39): `i5 modtime` after each full level; keep it per level so that the next entry can use `LEVELWARPMOD` (30) or `ADJACENTLEVEL` with a cached board.

Lab capture of a real 2.22 client crossing gmap edges, 2026-10-24 [cap]

Setup: a 2x2 gmap of open-grass levels; the client walked east, south, west and north across all four edges. - **Login.** The server sends `PLO_PLAYERWARP2` (49) with x*2*, *y*2, z+50, gmap cell x, gmap cell y, then the **gmap name** (not a level name). Then `PLO_LEVELNAME` (6) with the level, a second `6` with the gmap file name, 174, 10, and `156 SETACTIVELEVEL` = the gmap name. The `.gmap` file arrives as a file transfer (rawdata + file) because the client asks for it. The first big server frame also carries the neighbour levels' boards. The client sends `PLI_ADJACENTLEVEL` (35, `i5 modtime` + name) at login for the three neighbour levels (modtime 0 on a cold cache). - **A crossing uses only normal props; there is no level-warp packet.** - The client sets the flags `gr.x` / `gr.y` (`PLI 18`, flag set) on every half-tile step. They carry the **global** position: local + 64 * cell, so they run past 63.5. (Earlier notes in `version-profiles/2.22-2.31.md` say these flag sets come from the server's movement NPC rather than from client code; both fit if the server-sent movement weapon runs inside the client and the client sends the resulting flag sets. The capture cannot tell which.) - East: in the frame where global x reaches 64.0 the client sends `PLI 39` (the tampercheck constant, see 08), `gr.x=64`, and a props message with local X = 0 and `GMAPLEVELX` (prop 43) = 1. - South: the frame at global y 64.0 carries `PLI 39` and the flag only; the props message with local Y = 0.5 and `GMAPLEVELY` (prop 44) = 1 follows in the next half-step frame, because position props go out only on whole-tile steps. - West: `PLI 39` goes out at global x 63.5; the next props message has X = 63.0 and prop 43 = 0. North is the same with Y = 63.0 and prop 44 = 0. - Rule: local = global - 64 * cell; the cell prop changes in the same props message as the wrapped coordinate; position props use whole-tile steps; **2.22 re-sends `PLI 39` on each crossing**. - **Server answer to each crossing** (one frame): `6` new level name, `39` (server to client level mod time, only when the client does not have the level cached), `0` (an empty board packet), `6` gmap name, `174`, `10`, `156` gmap name. A second visit to a level omits the server's `39`. - The merged Seihai, connected to this lab, treated the gmap name in `PLO_PLAYERWARP2` as a level name and requested `<gmap>.nw`, which does not exist. Gmap support (this spec) is still not implemented; the above is the data it needs.

Seihai today

- Profile flag `levels.gmap` exists (set per version profile in `profile.rs`; check which ones before relying on it); the `LEVELNAMES` text is not parsed. PLO 49, 171, 172, 156, 39, 6 (LEVELNAME) is handled; PLI 35 is never sent. The rest are not implemented. 156/39/171/172 are counted as unhandled in the public-server report.
- `draw_minimap` draws a 64x64 level minimap (not the gmap).
- Single-level warps (`PLAYERWARP` 14, `send_levelwarp`) work.

Steps for Claude

1. Add `formats::gmap` parser: `GRMAP001`, WIDTH, HEIGHT, LEVELNAMES rows (quoted CSV, lower-cased lookup), MAPIMG/MINIMAPIMG, flags. Test with a hand-written 2x2 file.
2. Handle PLO 49 (decode the layout above) to enter a gmap; store `gmap_name`, `cell_x`, `cell_y`. Handle PLO 6 on a gmap as "the map name". Store 156 and 39 (modtime per level).

3. Fetch the `.gmap` with WANTFILE if absent, then the levels of the 3x3 neighbourhood: send `ADJACENTLEVEL` (35) for each neighbour.
4. Render neighbour levels offset by 64 tiles; compose collision across seams. Position in world = cell*64 + local.
5. On crossing an edge, send `PLPROP_GMAPLEVELX/Y` (43/44) plus the wrapped X/Y in one prop message. Let the server answer with the new level data; keep the old level data for one tick to avoid flashing.
6. Parse 171/172 into settings (file names + default x,y); fetch images only if a map screen exists. Otherwise ignore.
7. Gate everything on `profile.levels.gmap` so 2.05 keeps working.

How to test

- Lab: copy a 2x2 set of lab levels, write `lab2x2.gmap`, set `gmaps = lab2x2.gmap` in a **copy** of the lab `serveroptions.txt`, restart the lab gserver, log in with a 2.22 client (Seihai `--version 2.22`), warp into one of the levels and walk across an edge. Confirm PLO 49, 6, 156 and the neighbour level packets in the decoded frames. Display :9 only.
- Unit tests for the parser and for the cell/world coordinate maths.

NEEDS

~~a lab capture of a real 2.22 client crossing a gmap edge~~ Done 2026-10-24 (see above; 2.22 has no X2/Y2 props). ~~a 2.31 crossing~~ Captured in the lab 2026-10-24 (batch b2, seven level entries): 2.31 sends `PLI_ENTERLEVEL` (46, `GCHAR 1`) first, then the props message with the cell prop (43 or 44) and the wrapped local coordinate, and does not re-send `PLI 39` at the crossing; the server's answer matches 2.22 (`../decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`). Still open, thin evidence: the Z2 prop at a crossing, and how the 2.17+ client lays out the gmap overview on the map screen.

`client-internals/feature-specs/07-udp-peer.md`

07 UDP peer channel

Snapshots read: Graal_analysis `8da6800` (`protocol/udp-p2p.md`, `packet-layouts.md`, `tools/lab-setup.md`), Seihai `06aacfd` (`net/udp.rs`). Folded here from the public-server report: **PLO_HASNPCSERVER 44**, **PLO_ISLEADER 10** (as it relates to TCP fallback).

What it is

A client-to-client UDP channel in 2.05. The **server never sees it** (GServer-v2 has no implementation); the gserver only relays each player's address (props 30 IPADDR i5 and 31 UDPPORT i3) in `PLO_OTHERPLPROPS`. Layout: datagram = `seq` byte + GCHAR UDP id + fields, all XORed with `recipientId XOR 1`. No zlib, no length prefix. Receiver accepts only from a known peer ip:port, drops duplicate seq. Ids: 0 move, 10/11 bomb add/del, 12/13 arrow/fire spy, 14 explosion, 15 showimg, 16/17 horse, 18 throw carried, 19/20 item add/del, 21 remote hit, 22 chat, 23 level name (receive only), 24 entered-level (x*2*, *y*2, level) to the 3x3 gmap neighbourhood, 25 hit objects, 26 trigger action. Payloads for 0, 15, 21, 23, 24 are in the note.

Routing rule 2.05

A level action goes UDP-only when every other player in the level has a UDP port; otherwise it goes by UDP to peers that have one **and** by TCP to the server. Player props still go over TCP. **GServer-v2's `PLO_HASNPCSERVER` (44)**: if sent, 2.05 sends the TCP copy too.

Consequence for Seihai: UDP is optional. If Seihai advertises no UDP port (prop 31 = 0 or absent), 2.05 peers send every combat/object message over TCP, and Seihai receives them as normal PLO packets. UDP only saves latency. Seihai's profile has `udp` and `udp_off_by_default` flags already.

Seihai today

- `net/udp.rs`: `encode_udp`, `decode_udp`, `udp_key`, id constants, socket bind helper; lab-verified with 2- and 3-player pcaps (`examples/udp_replay`).
- Not wired into the session loop: no socket is polled, no props 30/31 are sent, UDP id handlers do not exist.
- PLO 44 and 10 are unhandled.

Steps for Claude

1. Keep UDP off by default for all profiles (document that peers fall back to TCP). Do **not** send prop 31 unless the user enables it.
2. Optional path, only when enabled: bind (`fixedudpport` or ephemeral), send props 30/31 once after login, poll the socket every frame. The XOR key is `recipientId XOR 1`: to send to peer P use `udp_key(P)`; on receive, use `udp_key(own player id)`.
3. Map each UDP id to the same internal event as its TCP twin (bomb 10 = PLO 11, etc., see 02) so there is one code path.
4. Send id 0 movement only when the position changed; fields X*2*, *Y*2, SPRITE, STATUS, CARRYSPRITE, animation letter (first letter of the gani name, upper or lower case as seen: `W`, `I`, `S`). Do not send id 23.
5. Store HASNPCSERVER (44) and ISLEADER (10) flags.

How to test

- `examples/udp_replay` against the existing pcaps (private, never committed; they contain the lab password in plaintext).
- Lab: two Wine 2.05 clients plus Seihai with UDP disabled; confirm bombs arrive over TCP at Seihai. Then enable UDP in Seihai and confirm datagrams decode.

NEEDS

nothing to implement. Open questions that do not block: which client version sends UDP id 23, and the meaning of the last byte in id 0 beyond "first letter of the animation" (needs a 2.22/2.31 UDP capture if those versions are to be supported; 2.22 defaults to `noudp=true`).

`client-internals/feature-specs/08-tampercheck.md`

08 Tampercheck (PLI 39) for 2.17+ and the process list

Snapshots read: Graal_analysis `8da6800` (`protocol/client-versions.md`, `version-profiles/2.22-2.31.md/.toml`), Seihai `06aacfd` (`protocol/profile.rs`), GServer-v2 (`TPlayer.cpp`

`msgPLI_MAPINFO`, `msgPLI_PROCESSLIST`, `TPlayerLogin.cpp`), the public-server report. Folded here: **PLO_LISTPROCESSES 182** and PLI 44/45 process list.

Naming trap

GServer-v2's older header calls PLI 39 `PLI_MAPINFO` ("might be gmap related"). The Graal_analysis notes call it `PLI_TAMPERCHECK`. **Client to server 39** is not **server to client 39** (`PLO_LEVELMODTIME`, see 06). Never mix them.

What it is [bin][cap]

- New in 2.17; absent in 2.05 and 1.x. Sent right after `PLI_LANGUAGE` (37) at login: it is the second message of the client's first gen-3/gen-5 frame, after `PLI 37 English`.
- Payload: **10 raw bytes, not two GINT5 words** (corrected 2026-10-24 from real-client lab captures). Bytes below 0x20 occur in the 2.17 and 2.22 values, which a GINT5 byte can never be, and no value contains 0x0a, so it needs no escaping on the line-framed wire.
- It is a **per-build constant**: the same 10 bytes at every login, for different accounts and different login keys.

Build	Token	10 bytes (hex)	Re-sent
2.17	`GNW22122`	`2a7bad195e231243d82c`	no (once at login; nothing more in about 90 s)
2.22	`GNW03014`	`250e4d0f962b03ede631`	once per gmap edge crossing; no periodic send in about 100 s
2.31	`GNW28015`	`291655b7602e87c9eb82`	about every 3.2 s

The same table is in `version-profiles/tampercheck-values.tsv`. The 2.22 value matches the earlier TOML record, so that record is confirmed. - 2.31 only: it also sends `PLI_SHOWIMGPLAYER` (24) with a 1-byte payload (value 9 seen) about every 5.1 s, and `PLI_ENTERLEVEL` (46, 1 byte, value 1) on every level entry (login and each gmap edge crossing; it does not come with a re-sent 39, which 2.31 sends on its 3.2 s timer only). Neither was in the profile notes; the meaning of the id 24 payload is unknown. (Lab batch c: 2.22 also sends 24 = 9 about every 5.1 s while another player is in its level, 2.31 always; 2.31 sends 46 on doors, links and server warps too and re-sends no 39 there. 2.22 sends no 46 but sends 39 in the same frame as each client-initiated level-link request, not after a server-side warp: `../decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`.) (Id 46 is settled: see the settled index in `README.md` and `../decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`; door and warp entries untested.) - **What the bytes are (decompile pass, 2026-10-24; all six lab values reproduced offline) [bin]:** two 32-bit words A then B, each written as 5 bytes in a form that is **not** GINT5 and **not masked**: byte 1 = `((v >> 28) & 0x0F) + 0x20`, bytes 2–4 = `((v >> 21) + 0x20) & 0xFF`, `((v >> 14) + 0x20) & 0xFF`, `((v >> 7) + 0x20) & 0xFF`, byte 5 = `(v & 0x7F) + 0x20`. Only bytes 1 and 5 are range-limited, which is why bytes below 0x20 occur. Both words are standard reflected CRC-32 (polynomial `0xEDB88320`, start `0xFFFFFFFF`) over the client's **own memory image** (the running exe), chained: the first call ends with the usual final inversion and its result seeds the next call, and the last call of each word skips the final inversion. Word A covers 8–9 contiguous ranges of the code section; word B covers one large code range plus one block of static data (a table of string pointers). Nothing depends on the login key, account, level or time, so the result is a per-build constant that changes only if the exe bytes change. Words per build: `version-profiles/tampercheck-words.tsv`; full write-up: `client-internals/decomp-findings-2026-10-24-b/01-tamper-check-id39.md`. - **2.31 re-send timing [bin]:** a 50 ms main-form timer keeps a counter modulo 64 (the length of the 64-character alphabet string it uses), so one cycle is 64 × 50 ms = 3.2 s. Phases: 1 anti-debug probe and a 5000 ms timer is armed; 10 recompute word A; 20 scan top-level windows against an obfuscated blacklist (not decoded); 30 recompute word B; 31 send PLI 39. The 5.1 s PLI 24 cadence fits the 5000 ms timer (inference). 2.17.1 has no such timer and 2.22 re-sends only from its level-change routine. See `client-internals/decomp-findings-2026-10-24-b/03-periodic-ids-2.31.md`.

Exact server behaviour [srv]

`TPlayer::msgPLI_MAPINFO` (id 39) reads the rest of the packet and returns. **GServer-v2 does not validate it.** The lab therefore cannot tell you whether a different value would be rejected. Real servers may check; unknown. Process list: the server sends `PLO_LISTPROCESSES` (182) once during login (2.17+). The client can answer `PLI_PROCESSLIST` (44) with a list of running programs (inferred from the names and the server's parse, not captured); `msgPLI_PROCESSLIST` parses it into a vector and ignores it. A later copy of the server also has 45 (process running). Anti-cheat only. Also relevant: `msgPLI_NULL` counts unknown ids and disconnects after more than 5, so never send ids that are not in the client's profile `pli` set.

Public servers

The 2026-10-24 run logged in to 4 public servers with 2.22 and **no id 39**; none disconnected within 20 s. That shows presence is not enforced at login on those four, not that it is never enforced. A real 2.31 client resends it every ~3.2 s, so a periodic check is plausible there, but no server behaviour has been observed.

Seihai today

- Profile flag `tampercheck` exists. `send_post_login` sends LANGUAGE only (no id 39), per the report. In the lab (2026-10-24) the merged Seihai logged in as 2.17, 2.22 and 2.31 without sending it, and GServer-v2 accepted that.
- 182 and 44 are not handled.

Steps for Claude

1. The bytes are a constant per unmodified build, so replay the table values; do not try to recompute them at run time.
2. In the 2.17/2.22/2.31 profiles add `tampercheck_bytes: Option<[u8;10]>` from the table above; send `PLI 39` right after LANGUAGE when the profile has `Some`. Version-specific re-sends (2.22 on each gmap edge crossing, 2.31 about every 3.2 s) only matter if a server is seen to enforce it.
3. Make it opt-in at first (`--send-tampercheck`) so the public-server default stays "same as the 4 servers that accepted us".
4. Handle 182 by sending nothing (or an empty `PLI_PROCESSLIST` if a server complains); log it.
5. Add a test asserting that the message after LANGUAGE in the first post-login frame is id 39 followed by exactly the 10 raw profile bytes.

How to test

- Lab with GServer-v2: confirm the extra packet causes no error and no invalid-packet counter increase (the handler exists). Compare Seihai's frames with the 2.22 capture frame by frame (`examples/lab_replay`).
- Cannot test enforcement locally.

NEEDS

- ~~For enforcement only: the 2.22 id-39 value builder~~ Done 2026-10-24: the decompile shows how the bytes are built (above). Computing them for a patched exe needs that exe's own code ranges, which are inside each client binary; replaying the per-build constant is enough for an unpatched client.
- The decompiles only show the sender. There is no check on the client's receive side, so whether any server enforces the value is still unknown.

- ~~For 2.17 and 2.31 values~~ Done 2026-10-24: captured from the real clients under Wine in the loopback lab (table above).

`client-internals/feature-specs/09-options.md`

09 In-game options

Snapshots read: Graal_analysis `8da6800` (`client-internals/graalconfig-reference.md`, `config-and-input.md`), Seihai `06aacfd` (`settings.rs`, `bin/main.rs`).

What it is

The 2.05 client has a **More Options** dialog, a **Player List Options** dialog and a **Controls** frame. All of them read and write one text file, `graalconfig.txt` (`key=value`, case-insensitive keys, rewritten in full with every key in fixed order whenever either dialog closes). Nothing goes over the network. Registry values hold the nickname, account list and (encrypted) passwords.

Parse rules (matter for a compatible reader) 2.05

- Booleans are true only if the lower-cased value is exactly `true`; a **missing** boolean reads as false.
- Integers: parse error keeps the default. Strings: used only if non-empty.
- Key binding: a key already bound earlier in the file becomes unbound; binding a key held elsewhere swaps it.

Keys (file order, default)

`waitvertical` false, `musicon` true, `usewinamp` false, `soundfxon` true, `reversestereo` false, `saveimages` true, `dontsavepms` false, `compabilitydraw` false (spelling as in the client), `nodecompressionerror` false, `head` (default `Fung Si Yun`), `joystickon` true, `nodirectinput` false, `joyselected`, `keyset_<action>_joystickbutton`, `keyset_<action>_key0..2`, `language` (English), `font` (Tempus Sans ITC), `fontsize` 24, `fixedport` false, `fixedudpport` 14899, `noudp` false, `nomassmessages` false, `nohtmlinpms` false, `limitnicknames` false, `nicknamelimit` 50, `dontconnectlevels` false, `notoalls` false, `nohtmlimages` true, `sortplayersbyname` false, `dontloadlistheads` false, `dontsavepasswords` false, `dontusedirectx7` false, `fullscreenmode` 1024x768. Actions with three key slots: up, left, down, right, item, attack, action, map, switchtochat, itemlist, pause. The per-action default keys for 2.05 are not listed in the notes; the fixed keys of 1.37 were arrows or IJKL, D item, S or Space attack, A action, M map, Q item list, P pause, Tab chat (`versions-133-139.md`). Default joystick buttons are in `graalconfig-reference.md`. Later clients add `midivolume`, `mp3volume`, `radiovolume`, `sfxvolume`, `voicevolume`, and an Audio/Microphone tab (2.17+).

What the options change at run time

`musicon`/`soundfxon`/`reversestereo` (01); `noudp`/`fixedport`/`fixedudpport` (07); `nomassmessages`, `notoalls`, `nohtmlinpms`, `nohtmlimages`, `sortplayersbyname`, `dontloadlistheads` (14, player list); `dontconnectlevels` stops neighbour-level preloading (06); `limitnicknames` truncates displayed nicknames; `saveimages`; `language`.

Seihai today

- `settings.rs` is Seihai's own launcher file (assets path, account, password, version, last server, listserver, offline flag). It is **not** `graalconfig.txt`. It stores the account password in the settings file; note that 2.05 encrypts saved passwords and has `dontsavepasswords`. Decide with the user before keeping plaintext.
- `bin/main.rs` has fixed keys (arrows move, A/Space interact, Enter chat, F2/F3/F4/F5/F9/F12 dev actions, PageUp/Down scroll). No options screen. The launcher `gui.rs` (egui) picks assets and server.

Steps for Claude

1. Add `options.rs`: a struct with the keys above, `load_graalconfig(path)` implementing the parse rules, `save_graalconfig(path)` writing all keys in order. Keep Seihai-specific keys in `settings.rs`.
2. Round-trip test: parse a 2.05-style file with a missing boolean and assert the false-on-missing behaviour; write and re-read.
3. Key map: a table from action to up to three key names; implement the "already bound is unbound" load rule. Replace the hard-coded keys in `main.rs` with lookups.
4. In-game screen: an egui or framebuffer window opened with F1 (original key) listing music, sound, reverse stereo, UDP options, nickname limit, no-toalls, player-list sort. Apply without restart where easy.
5. Wire only options whose feature exists (sound after 01, UDP after 07, gmap after 06). Others are stored and written back unchanged.
6. Optionally read an existing `graalconfig.txt` from the assets folder as defaults.

How to test

- Pure unit tests (file parse/write). No lab needed.
- Compare against a real file written by the 2.22 client in the lab (the notes say 2.22 uses the same keys in the same order).

NEEDS

nothing.

`client-internals/feature-specs/10-right-click-menu.md`

10 Right-click context menu (player list popup, profile)

Snapshots read: Graal_analysis `8da6800` (`versions-133-139.md`, `version-profiles/141-217.toml`, `protocol/chat-pm-rc.md`, `client-architecture-205.md`), Seihai `06aacfd`, GServer-v2 (`TPlayerRequestText.cpp`, `TPlayer.cpp` PM/profile handlers). Folded here from the public-server report: **PLO_STAFFGUILDS 47** (player-list "Staff" section).

What is known [2.05][inf]

- There is no context menu on the wire. It is client UI. The 2.05 client has a `TPlayerList` form (F7) with a popup menu. **[2.05][dec]** The 2026-10-24 decompile gives the item order: **Message (default item), History, Buddy, Ignore, Profile**. The 1.33 build had Message, Ignore, History; Buddy came in 1.35 (`buddyaccounts.txt`); Profile and the Player List Options window came in 1.39. "Right-click profile" is a listed feature in the 2.17 profile.

- **`TPersonAction` is not a player menu.** It is the level editor's "Set Npc's Action" script editor (opened from the NPC editor, see `level-editor-205.md`). There is no per-player action form (trade, duel and so on) in 2.05. An earlier version of this spec listed it as an unexplained 2.05 form; that is corrected.
- Window facts: the list is a stay-on-top form with a multi-select list view; a panel caption reads "--- N online ---"; buttons Profile and Options. Right-click selects the row under the cursor, then sets the Ignore and Buddy check marks to "checked if every selected player is in that list". Double-click or Enter opens the read-PM window if a PM from that player is pending, otherwise the compose window. PM window titles are "Send Message" and "Read Message". The Options window has 5 check boxes; the profile edit window has 10 fields.
- Actions behind the items, with what is known:
  - **Message**: opens the PM box for **all selected rows** (an existing PM window for the same set is reused); sends `PLI_PRIVATEMESSAGE` (28) `s count, s player...` + text. Recipient encoding in 2.05: a two-byte count, then two bytes per player id, **at most 50 recipients**. Done in Seihai (`send_pm`).
  - **Profile**: uses only the **first** selected player and, when online, sends `PLI_PROFILEGET` (80) with the account; the listserver answers `PLO_PROFILE` (75) with 10 length-prefixed fields; fields 1-9 are saved locally. Edit own profile with `PLI_PROFILESET` (81). GServer-v2 does **not** answer profile requests; the listserver does.
  - **Ignore**: local only. Toggles each selected account in `ignoredaccounts.txt` (1.34+: by account name) and saves the buddy list too; ignored players are drawn red in the list (1.34+) and their chat and PMs are dropped by the client. The client holds an auto-reply string "Your messages are ignored."; the condition under which it is sent was not traced.
  - **History**: opens the message history window (not traced further).
  - **Buddy**: local `buddyaccounts.txt` in 2.05, toggled for each selected account; **Ignore and Buddy are mutually exclusive**; see 14 for 2.17+.
- Player list sections: 1.33 had GPs, Guild, Normal, Ignore. 2.14+ lists staff in a section named from `PLO_STAFFGUILDS` (47): a list of guild names that count as staff (GServer `staffguilds` option, sent at login by `sendLogin`).
- Right-click on a player's sprite in the world: **no such handler was found in the 2.05 decompile**. The only player popup is the F7 list's.

Seihai today

- `ui.rs draw_player_list` shows the other players; `nearest()` picks the closest. No menu, no mouse handling beyond the egui launcher. `PLO_STAFFGUILDS` (47) and `PLO_PROFILE` (75) are not handled; `send_pm` exists.

Steps for Claude

1. Add mouse input to the framebuffer loop (minifb gives `get_mouse_pos`, `get_mouse_down`). Right-click inside the F7 list selects a row and opens a small menu drawn with `ui.rs` primitives, in the 2.05 order: Message (default, bold), History (hide until it exists), Buddy, Ignore, Profile. Support multi-select for Message; set the Buddy and Ignore check marks as above and keep them mutually exclusive. Double-click or Enter opens the PM compose box.
2. Message: open the existing chat input in PM mode with the target prefilled.
3. Profile: send `PLI_PROFILEGET` (80) and render the reply when PLO 75 arrives (implement the 10-field parser from `chat-pm-rc.md`; drop field 0).
4. Ignore: keep a set of account names, saved to `ignoredaccounts.txt` next to the settings (one account per line). Filter chat/PM/toall/bubbles for them and tint the row.

5. Buddy: see 14.
6. Store 47 and draw a "Staff" section label for players whose guild matches.
7. Do not add a menu for a world click: 2.05 has none.

How to test

- Headless unit tests on the menu model (selection, ignore filter). Visual check under Xvfb on display :9 only.
- Lab: with a lab account PLI 80 reaches GServer, which sends nothing back (expected); profile display needs a listserver reply, so test with a recorded 10-field PLO 75 frame in `lab_replay`.

NEEDS

Filled by the 2026-10-24 2.05 decompile: item order, check-mark rules, PM recipient encoding, what `TPersonAction` is, and that no world right-click exists (all above).

Still open (none blocks a menu): - The 1.39 popup items (its menu strings are built character by character and were not re-read). - Where the "Your messages are ignored." auto-reply is sent from. - What the History window shows (not traced). - Enable/disable (greyed) rules beyond the check marks, if any.

`client-internals/feature-specs/11-emotes-ganis.md`

11 Emotes and non-walk GANIs

Snapshots read: Graal_analysis `8da6800` (`formats/gani.txt`, `protocol/udp-p2p.md`, `packet-layouts.md`, `versions-133-139.md`, `player-props.md`), Seihai `06aacfd` (`assets/gani.rs`), GServer-v2 (`msgPLI_SHOWIMG`, `TPlayerProps.cpp`).

What it is

Two separate things.

A. Animation state

- **2.1+ (incl. 2.22, 2.31)**: `PLPROP_GANI` (10) is a string gani name (`idle`, `walk`, `sword`, `spin`, ...). The receiver fetches `<name>.gani` with WANTFILE if it lacks it. GServer stores it and forwards it; when it is `spin` it fabricates four HITOBJECTS (02).
- **2.05**: prop 10 is the pre-2.1 bow form. The animation is in `PLPROP_SPRITE` (17): `sprite % 4` = direction and `sprite / 4` = a sprite number S. **[2.05][dec]** The 2.05 decompile (2026-10-24) gives the exact table, see "2.05 sprite prop" below. The UDP movement datagram has a trailing byte that looked like the first letter of the animation name (`W`, `I`, `S`); the client also keeps a table that maps gani-name substrings to one letter (`attack` A, `carry` C, `dead` D, `hurt` H, `idle` I, `kick` K, `lift` L, `ride` R, `sit` T, `spin` U, `swim` V, `sword` S, `walk` W, `def` X and a few more, used to shorten cached gani file names; full table in `../decomp-findings-2026-10-24/gani-short-codes.tsv`).
- Props that interact: CARRYSPRITE (19), STATUS bits, HORSEGIF (21), HEADGIF/BODYIMG, `PLPROP_GATTRIB1-30` (gani parameters `#P1..`).
- GANI file grammar (SPRITE, ANI/ANIEND frames, PLAYSOUND, LOOP, SETBACKTO, DEFAULTHEAD...) is in `formats/gani.txt`. Seihai's parser covers frames, loops and `PLAYSOUND`; effects and scripts are not run.

B. Emotes

- Emotes are **images shown over a player's head**. On the wire: `PLI_SHOWIMG` (24) client to server, rest passed through; GServer re-emits it as `PLO_SHOWIMG` (32) with `s player id` in front to the level area (`msgPLI_SHOWIMG`, `sendPacketToLevelArea`). In 2.05 the same payload also goes over UDP id 15. **[srv] [cap]**
- Layout seen: four GCHARs (index, x, y, name length) then the image name, for example the AFK emoticon `emoticon_afk_stay.mng`, re-sent about every 3 s while shown. Trailing bytes follow. **[cap]**
- Triggers: 1.38+ Ctrl+letter (A AFK, B BRB, Z Zzz stay until the player moves; others from D, E, F, G, H, L, M, O, P, R, S, T, W, X; 1.39 adds C, K, Y). The assets include an `emoticons` folder (29 png/mng files). 2.17+ also has chat-line emoticons.
- **Correction (2026-10-24 decompile of 2.05 rev2; the 1.39 build has the same routine).** The key path is **local only and sends no packet**. Ctrl+A to Ctrl+Z (no text field focused, player not dead or locked) lowercases the letter and walks the client's loaded file list for the **first file whose name starts with `emoticon_<letter>`** and whose extension is png, mng or gif (a prefix match: Ctrl+A matches `emoticon_afk_stay.mng` if it is the first `emoticon_a...` file in the list). The image is drawn at the player's x+2, y-5 (a little lower when a second flag is set) for **50 ticks (2.5 s)**. There is no letter-to-name table and no hard-coded AFK/BRB/Zzz string in the client: the mapping is whatever files exist. `PLI_SHOWIMG` (24) is sent **only by the script command `showimg`**, never by the key handler. The earlier wording of this spec (Ctrl+letter sends SHOWIMG) was an inference from the captured packets and is wrong for the key path.
- **Open:** how other players see a Ctrl+letter emote. The captured SHOWIMG frames (AFK image re-sent about every 3 s) are not explained by the key handler; they may come from a server-side NPC or script that calls `showimg`, or from a path not yet traced. Treat the emote *send* as unknown. (Lab batch c, 2.31 and 2.22: the stock server relays any client 24 payload unchanged as 32 and no client reacted to a 1-byte payload; the real 2.31 payload is a single byte 9 sent about every 5.1 s, not an emote form: `../decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`.)
- `PLO_SHOWIMGNPC` (166) shows images on NPCs.

2.05 sprite prop [2.05][dec]

Derived from the 2.05 rev2 decompile; tables are in `../decomp-findings-2026-10-24/` (`sprite-step-table.tsv`, `gani-to-step.tsv`, `gani-name-table.tsv`, `status-bits-prop18.tsv`, `player-modes.tsv`). - Byte on the wire = 32 + direction + 4 x S (the 32 is the usual GCHAR offset, so the decoded prop is `dir + 4 x S`, matching `sprite % 4` and `sprite / 4` above). - S comes from the current animation step through a step table, and **only while the active gani is the default `def` gani and the step is 38 or lower**. Other ganis do not change the sprite number. The sender writes 0 for names it does not know. A hurt player is special-cased: S = 32 (decoded `0x80 + dir`). - Step to S (steps 0 to 38): `0, 4, 5, 4, 3, 2, 1, 2, 3, 6, 7, 8, 9, 10, 11, 12, 13, 11, 12, 14, 15, 16, 17, 23, 18, 19, 20, 19, 18, 21, 22, 21, 18, 24, 25, 26, 27, 28, 29`. Steps 9 to 13 (S 6 to 10) are the sword swing and the only ones that can hit (see 02). - Gani name to step rule: idle and def = frame; walk and walkslow = frame+1; kick 9; sword, ridesword, spin = 9 + (frame mod 5); push 14+frame; grab 19; pull 20+frame; lift 23; carrystill and hiddenstill 24; carry and hidden 25+frame; shoot 33; ridestill 34, ridefire 35, rideeat 36; ride 35 + (frame and 1); sit 37; sleep 38; hurt and ridehurt 39; dead 40 (not sent). - **Swim, lava and dead are not on the wire.** All three are sent as sprite 0 (idle) plus direction. Swim is a local mode (10) entered on water tile type 11 (with a splash); lava (tile type 12) shows the local `lava` gani. The dead state is carried only by the STATUS prop dead bit (see 04). There is **no swim sprite**, so a 2.05 client cannot tell a swimmer from an idle player by prop 17. - The 31 built-in gani names (index 0 to 30): idle, walk, walkslow, kick, sword, ridesword, spin, push, grab, pull, lift, carrystill, hiddenstill, carry, hidden, shoot, ridestill, ridefire, rideeat, ride, sit, sleep, hurt, ridehurt, dead, maps1, maps2, maps3, swim, lava, carried. - Caveat: the "only for

def" condition was read from the sender and is recorded as observed, not confirmed on the wire. - STATUS prop (18) bits: 0x01 editor or paused, 0x02 hidden (mode 14 or hidden timer), 0x04 male, 0x08 dead (see 04), 0x10 weapons allowed, 0x20 sword hidden (the hidesword timer, ../status-bit-0x20.md), 0x40 has charged spin.

Seihai today

- Parses gani files and keeps a walk animation player (set_walk_gani, walk_frame). Does not send SPRITE animation indexes beyond walking, does not draw other players' non-walk animation, and has no emote send/receive (PLO 32 not handled). Other players are drawn from props.

Steps for Claude

1. For 2.22+ profiles: on movement/attack/lift state changes send PLPROP_GANI with the gani name; draw other players using the gani named in their props, loading it via the asset path or WANTFILE.
2. For 2.05: send prop 17 as dir + 4 x S using the step table above (idle = S 0, walk = frame+1 mapped through the table, sword swing S 6 to 10, and so on). Send sprite 0 plus direction while swimming and while dead (no swim sprite exists). To draw other 2.05 players, invert the table: S -> pose, with the caveat that several steps share an S. Do not invent a swim sprite.
3. Implement PLO_SHOWIMG (32): decode, spawn a timed overlay above that player (x,y given in the packet, name from the packet), expire after about 3 s unless refreshed.
4. Emote key (2.05 behaviour): on Ctrl+letter, find the first file in the loaded asset list whose name starts with emoticon_<lowercase letter> and has a png, mng or gif extension; draw it locally at the player's x+2, y-5 for 50 ticks (2.5 s). **Send nothing.** Send PLI_SHOWIMG (24) only from a script showimg command, with the captured field layout. Whether other players should see a key emote is open; if Seihai wants that, treat it as a Seihai choice (sending SHOWIMG is protocol-legal) and mark it so.
5. Draw .mng as a first frame only (GIF/MNG decoders are not in Seihai); skip PNG variants gracefully.
6. Add the hurt, dead, sword, lift, grab, push poses as the combat/inventory/death work (02-04) needs them.

How to test

- Lab: Wine 2.05 client sends an AFK emoticon (Ctrl+A) next to a Seihai session; check PLO 32 arrives and decodes. Replay the existing two-player capture through SessionState for ids 15 and 24.
- Load every .gani in the lab world ganis folder with Gani::parse as a smoke test.

NEEDS

Filled by the 2026-10-24 2.05 decompile: the sprite-index to animation table and the Ctrl+letter lookup (both above). 2.22+ needs nothing: names are plain strings.

Still open: - How other players see an emote (the key path sends nothing; see the correction above). A lab capture of a real 2.05 client pressing Ctrl+A next to a second client would settle it. - Swim on the wire: none found in 2.05. Whether 2.22 sends a swim gani name (prop 10) is covered by the 2.1+ notes, not by this decompile. - Some mode values (see ../decomp-findings-2026-10-24/player-modes.tsv). - 1.39 sprite and gani behaviour was not re-read (the emote routine is the same).

client-internals/feature-specs/12-motd-rules.md

12 Server MOTD and rules

Snapshots read: Graal_analysis `8da6800`, Seihai `06aacfd`, GServer-v2 (`TPlayerLogin.cpp` `sendLoginClient`, `TServer.cpp` message load, `CWordFilter.cpp`, `TPlayerProps.cpp`), the lab server config. Folded here from the public-server report: **PLO_STARTMESSAGE 41, PLO_RPGWINDOW 179**, and the empty `PLO_SERVERTEXT 82`.

Packets (server to client, at the end of login) [srv]

Id	Name	Payload	Versions
41	STARTMESSAGE	the server message as one text string	1.x-2.x; 2.05 has it (client form `TServerMessage`); newer 5/6 clients ignore it
179	RPGWINDOW	quoted, comma-separated lines: `"Welcome to <server name>.","<vendor> <app> programmed by <credits>."`	2.1+ (in the 2.17/2.22 sets)
82	SERVERTEXT	empty at login; the server sends it "to allow serverwarp and some other things". Later it carries replies to `PLI_REQUESTTEXT`/`SENDTEXT` as `weapon,type,option,data` tokens	2.1+
171 / 172	BIGMAP / MINIMAP	see 06	2.1+

`sendLoginClient` order: level data (warp), bigmap, minimap, RPG window, start message, empty server text, flush.

The message text (`TServer`) [srv]

Loaded from `config/servermessage.html`, then every `\r` is removed and every `\n` replaced by a space, so it is **one long line of HTML**. The lab file is the default HTML sample (head, style, a `div` with a welcome and a changelog block). The client is expected to render HTML or at least strip tags.

"Rules" in GServer

`config/rules.txt` is the **word filter**, not a player-facing rules text. Syntax: optional `WARNMESSAGE`, `SHOWWORDSTORC`, then `RULE` ... `RULEEND` blocks with `CHECK` (pm, chat, nick, toall), `MATCH` (`?` wildcard), `PRECISION` (number or percent), `WORDPOSITION` (full, start, part), `ACTION` (log, tellrc, replace, warn, jail, ban) and an optional per-rule `WARNMESSAGE`. `CWordFilter::apply` runs on chat (prop 12), PM (`msgPLI_PRIVATEMESSAGE`), toall, and nickname changes. `replace` swaps matched letters for `*`; `warn` replaces the whole message with the warn text; `log`/`tellrc` go to server log and RC; **`jail` and `ban` are stubs and do nothing**. For chat, if the text was changed the server echoes the new CURCHAT to the sender so the client's own bubble shows the censored text. Real "rules" a player sees come from the server message, the RPG window and NPCs.

Seihai today

- 41, 179 and 82 are counted as unhandled. Chat text arrives and displays; a censored echo from the server is just another prop update.

Steps for Claude

1. Handle PLO 41: store the raw text in `SessionState.start_message`.
2. Small HTML reducer: drop `<style>`, `<script>` and comments blocks, convert `<br>`/`</p>`/`</div>` to newlines, strip other tags, decode a few entities, and wrap to the chat width. Show it in the existing panel (`ui::draw_panel`) once after login, dismissed with Enter/Esc. Unit test with the lab `servermessage.html`.
3. Handle PLO 179: parse the quoted list and print the lines to the chat log.
4. Handle 82 with empty payload as a no-op marker (`server_text_ready = true`); log non-empty payloads for later use (14).
5. Offer a `--no-motd` flag for scripts and tests.
6. No filter logic is needed in the client.

How to test

- Lab: edit a **copy** of `servermessage.html`, restart the lab gserver, log in with Seihai, and assert `start_message` equals the file with newlines folded. Add a rule in `rules.txt` (`RULE` with `CHECK chat`, `MATCH`, `ACTION replace`), send a chat line, and check the CURCHAT echo contains `*`.
- Replay frames from the public report only as structure checks; the report notes all four public servers sent 41 and 179.

NEEDS

nothing blocking. Optional: "the 2.05 start-message handler" (does it show the HTML in `TServerMessage` immediately or only on request).

`client-internals/feature-specs/13-ban-kick.md`

13 Ban and kick handling

Snapshots read: Graal_analysis `8da6800` (`protocol/gserv-1999.md`, `packet-ids.md`), Seihai `06aacfd` (`protocol/dispatch.rs`, `net/mod.rs`), GServer-v2 (`TPlayer.cpp` login and cleanup, `TPlayerLogin.cpp` `sendLogin`, `TPlayerRC.cpp`, `TServer.cpp` `isIpBanned`, `TServerList.cpp`).

What it is

Every refusal, kick and ban reaches the client as **`PLO_DISCMESSAGE` (16)**: rest = text, `\r` = line break. The server then closes the socket (or the client does). Seihai already parses 16 into `disconnect_reason`.

Exact server behaviour (messages, in the order they can happen) [srv]

`TPlayer::msgPLI_LOGIN` (steps 1-4), then listserver verification (`TServerList`), then `TPlayer::sendLogin` (steps 5-8): 1. Player limit (`maxplayers`): "This server has reached its player limit." 2. IP ban (`TServer::isIpBanned`, wildcard match against `config/ipbans.txt`, skipped for staff with the modify-staff

right): "You have been banned from this server." 3. Client version not in `allowedversions.txt` (ranges `a:b` allowed): "Your client version is not allowed on this server." plus a line with the allowed list. 4. Listserver down: "The login server is offline. Try again later." Listserver rejected the login: the listserver's own text (for example wrong password), then disconnect. 5. Account banned (`BANNED 1` in the account file, not for staff): "You have been banned. Reason: <BANREASON>" (newlines become `\r`). 6. `onlystaff`: "This server is currently restricted to staff only." IP not in the account's `IPRANGE`: "Your IP doesn't match one of the allowed IPs for this account." 7. Account already online: if the old session was idle for 30 s or more, the old one gets "Someone else has logged into your account." and is dropped; otherwise the new login gets "Account is already in use." (the check is case-insensitive). 8. No start level: "No level available." While connected: - Idle: `disconnectifnotmoved` with `maxnomovement` seconds: "You have been disconnected due to inactivity." - More than 5 unknown packet ids (`msgPLI_NULL`), or garbled props: "Disconnected for sending invalid packets." - RC kick (`msgPLI_RC_DISCONNECTPLAYER`): "One of the server administrators, <name>, has disconnected you" plus " for the following reason: <reason>" if given. - RC ban by account (`msgPLI_RC_ACCOUNTSET` and `msgPLI_RC_PLAYERBANSET`): "<name> has banned you. Reason: <reason>" and the account's `BANNED`, `BANREASON` are saved. - RC account reset: "Your account was reset by <name>". - `PLO_UPGRADEURL` (169) can accompany a disconnect to turn Reconnect into an Upgrade button (2.14+). The word filter's `ban` action is not implemented (12).

Seihai today

- `SessionState.disconnect_reason` is filled; the CLI prints it. No classification, no UI, no retry policy. The public-server run saw clean logins only, so no refusal text has been captured from a public server.

Steps for Claude

1. Add `enum DisconnectKind { PlayerLimit, IpBanned, VersionNotAllowed, LoginServerOffline, ListserverRejected(String), AccountBanned(String), StaffOnly, IpMismatch, InUse, KickedByStaff, Inactivity, InvalidPackets, Other }` and `classify(&str) -> DisconnectKind` (match on the phrase starts above, case-insensitive; keep the raw text).
2. Show the reason in the launcher/GUI on exit with the raw text and a hint per kind (for example VersionNotAllowed shows the allowed list; InUse says wait 30 s).
3. Retry policy: never auto-reconnect after a ban, version, staff-only, IP mismatch or invalid packets; allow a manual retry for InUse, player limit, offline login server.
4. Keep the invalid-packet guard from 02/08: assert in tests that no frame Seihai sends has an id outside the profile's `pli` set.
5. Record the reason in the exit code of the CLI (nonzero, distinct per kind) for lab scripts.

How to test

- Lab (private display :9): set `maxplayers = 1` on a **copy** of the lab options and log in twice; put the placeholder account's `BANNED 1` and `BANREASON test` into a copy of the account file; add the loopback address to `config/ipbans.txt` copy; use an unlisted version string; log in the same account from two sessions; stop the listserver stub. Check the exact text and `classify`.
- Unit tests feeding each phrase to `classify`.
- No public server contact.

NEEDS

nothing. (The text of refusals from the real listserver for a banned or unknown account is not in the captures; capture it only if the user approves a public login.)

`client-internals/feature-specs/14-friends-ignore.md`

14 Friends (buddies), ignore, mute, status

Snapshots read: Graal_analysis `8da6800` (`versions-133-139.md`, `version-profiles/*.toml`, `protocol/chat-pm-rc.md`, `graalconfig-reference.md`), Seihai `06aacfd`, GServer-v2 (`TPlayerRequestText.cpp`, `TPlayerProps.cpp` status message, `TPlayerLogin.cpp`), graal-serverlist (`TServer.cpp` `msgSVI_REQUESTBUDDIES`). Folded here from the public-server report: **PLO_STATUSLIST 180**.

What it is

Four separate features.

Feature	Version	Where it lives
Ignore a player	1.33; by account name from 1.34 (`ignoredaccounts.txt`)	client-local; ignored players shown red and their chat/PMs dropped
Buddy	1.35 local `buddyaccounts.txt`; 2.17+ also through the listserver	local in 2.05; listserver in later builds
Mute	2.17+	`PLI_MUTEPLAYER` (43): `s playerId, c 1/0` (from the playerlist Mute item)
Status (Online/Away/DND/...)	2.14+	`PLO_STATUSLIST` (180) list, `PLPROP_PSTATUSMSG` (53) index

Additional ignore switches: `nomassmessages`/`notoalls` options and the ADDITFLAGS prop bits (`0x01` no mass message, `0x04` no toall, `0x08` voice off) (09).

Server behaviour [srv]

- **Ignore**: none. The server never learns about it.
- **Mute (43)**: GServer-v2 defines `PLI_MUTEPLAYER` in the enum but registers **no handler**. It falls to `msgPLI_NULL`, which counts as an invalid packet: **after more than 5 such packets the server disconnects the client**. A real 2.22 client does send it (see lab findings below), and two sends in a lab session did not disconnect.
- **Buddies (2.17+)**: the client sends `PLI_SENDTEXT` (the generic text packet) with tokens `GraalEngine, lister, verifybuddies | addbuddy | deletebuddy` (+ account list). `msgPLI_SENDTEXT` forwards those three options (not for guest accounts) to the **listserver**; graal-serverlist answers `msgSVI_REQUESTBUDDIES` with a `PLO_SERVERTEXT` (82) whose tokens are `weapon, lister, buddylist, <comma list>` built from a database query. GServer itself stores nothing.
- **Status**: `PLO_STATUSLIST` (180) at login (2.1+/RC): comma list from `playerlisticons` (lab: Online, Away, DND, Eating, Hiding, No PMs, RPing, Sparring, PKing). The client sets its own index with `PLPROP_PSTATUSMSG` (53); the server validates it against the list length and rebroadcasts it to everyone (`PLO_OTHERPLPROPS`).
- **Player list pulls**: `PLI_REQUESTTEXT` types `pmservers`, `pmserverplayers <server>`, `pmunmapserver`, `pmguilds` go to the listserver for cross-server and guild tabs. **PM** (`PLI_PRIVATEMESSAGE` 28) is already implemented.

Lab findings from a real 2.22 client, 2026-10-24 [cap]

- Player-list window sections: Staff, Buddies, Guild, Normal, Ignore. Right-click on a player gives: Message, History, Buddy (check mark when set), Ignore, Mute, Profile. (This is the 2.22 popup; the 2.05 popup is a different, shorter menu, see 10.)
- **Buddy add and delete send nothing** on the game-server or listserver connection. The client writes `buddyaccounts.txt` locally and deletes the file when it becomes empty. The `verifybuddies`, `addbuddy` and `deletebuddy` requests were **not** sent in this lab, so their token layout is still unseen. The assumption above that 2.17+ buddies go through the listserver is not supported by this client in this setup; it may only trigger for accounts the listserver knows (a database-backed listserver) or at some other moment.
- **Ignore** is local too: it writes `ignoredaccounts.txt`, one account per line, and sends no packet.
- **Mute** sends `PLI 43`: `GSHORT playerId` + `GCHAR 1` (mute) or `GCHAR 0` (unmute), one message per click. The list entry does not move on Mute, unlike Buddy and Ignore, which move it.
- Not captured: the listserver-side buddy list reply and any status change.

Seihai today

- PM and toall send/receive (`send_pm`, `send_toall`); chat shows. No ignore filter, no buddy list, no status UI. 180 is unhandled. `send_text` helpers for `PLI_SENDTEXT`/`REQUESTTEXT` do not exist.

Steps for Claude

1. **Ignore (all versions)**: `ignored: HashSet<String>` loaded from `ignoredaccounts.txt` (one account per line, lower-case compare), applied to chat bubbles, `PLO_PRIVATEMESSAGE` (37), toall. Needs the account name prop (34) for each player; show tinted in the F7 list (10).
2. **Buddy (2.05)**: `buddyaccounts.txt` the same way; mark buddies in the list and print a login notice when a buddy appears (`PLO_ADDPLAYER`/`OTHERPLPROPS` with PCONNECTED). Local only.
3. **Buddy (2.17+, lab)**: implement `send_text(weapon, type, option, args)` using the tokenized form from `TPlayerRequestText.cpp`; send `GraalEngine, lister, addbuddy, <acct>`; parse PLO 82 `buddylist`. Behind a flag until captured.
4. **Status**: parse 180 into the list; send `PLPROP_PSTATUSMSG` (53) when the user picks one (Away on a timer is original behaviour, off by default); show others' status icon index in the list.
5. **Mute (2.17+)**: client-local mute set (same filter as ignore). Optionally also send `PLI 43` (`GSHORT playerId`, `GCHAR 1/0`) as the real client does, but GServer-v2 counts it as an invalid packet, so keep it off by default against that server.
6. Tests: ignore filter on a scripted chat stream; buddy file parse; status prop round trip; assert no `PLI 43` is emitted.

How to test

- Unit tests for the filters (no network).
- Lab: two sessions, ignore one account locally and send chat from it (the second Seihai or the Wine client); status: set a status index and check the other client receives prop 53. Buddy via listserver: the lab stub has no buddy table, so extend the stub to answer `addbuddy`/`verifybuddies` with a `buddylist` text, or run graal-serverlist with its database.

NEEDS

the mute payload is now known (see the lab findings). Still open: a capture of the buddy requests (`verifybuddies`, `addbuddy`, `deletebuddy`) and the listserver's `buddylist` reply, which the real client did not send in the 2026-10-24 lab. That needs a database-backed listserver or another trigger. No decompile needed; 2.05 needs nothing (local files).

`client-internals/feature-specs/README.md`

Seihai laundry list: the 14 "Not yet discussed" topics

Written 2026-10-24 (Asia/Tokyo, JST). Sources checked: Graal_analysis `8da6800` (main), Seihai `06aacfd` (main, 2026-10-24 03:23 JST), GServer-v2 source (local checkout), graal-serverlist source, the offline lab, and the public-server report of 2026-10-24 17:56 JST (structure only; no addresses copied). Nothing was run against the lab or any public server for this pass. No decompiler output, addresses or auto-generated names appear in these files. GServer behaviour is described in my own words with function names only.

Sufficiency table

"Enough" means Claude can implement the topic against the lab with what is already written. Value = worth to a playable client (1-5). Effort = work (1-5, 1 = a few hours). Score = value / effort.

#	Topic	Existing material enough?	Value	Effort	Score	Rank	NEEDS
13	Ban and kick handling	Yes	3	1	3.0	1	nothing
12	Server MOTD and rules	Yes	3	1	3.0	2	nothing
04	Death, respawn, regen	Yes (server side and 2.05 client side, from the 2.05 decompile)	4	2	2.0	3	nothing; open: the "same name group" respawn-point rule, 1.39 death handler
08	Tamperchec k 39 (+ process list 182)	Yes (constants for 2.17, 2.22 and 2.31 captured; derivation decoded)	2	1	2.0	4	nothing. Derivation settled (two chained CRC-32 words over the client's own image; per-build constants for 2.17, 2.22, 2.31). Left: whether any server enforces it (cannot be settled offline)
03	Inventory, items, chests	Yes	5	3	1.7	5	nothing

#	Topic	Existing material enough?	Value	Effort	Score	Rank	NEEDS
14	Friends, ignore, mute, status	2.05 (local files) yes; 2.22 buddy/ignore local and mute payload captured; listserver buddy requests unseen	3	2	1.5	6	only if buddies should go through a listserver: a capture of `verifybuddies`/`addbuddy`/`deletebuddy` (needs a database-backed listserver or another trigger)
02	Weapons and combat	Yes for the protocol; 2.05 sword hit box and swing sender known from the decompile	5	4	1.25	7	nothing for interop; open: whether the attacker also sends HURTPLAYER (26) on a sword hit
06	GMAP overworld and map links	Server side, file format and 2.22 client crossing behaviour yes (lab capture); 2.31 crossing captured (lab, batch b2)	4	4	1.0	8	nothing for 2.22. 2.31 crossing now captured in the lab (cell prop plus wrapped coordinate, PLI 46 before it); left, thin evidence: the Z2 prop and the overview layout on the map screen
11	Emotes and non-walk GANIs	Yes (2.22+ names are strings; 2.05 wire sprite formula and Ctrl+letter lookup known from the decompile)	3	3	1.0	9	nothing to build; open: how other players see a key emote, swim on the wire (none in 2.05)

01 | Sound and music | Yes for WAV effects | 3 | 3 | 1.0 | 10 | nothing |
09 | In-game options | Yes | 2 | 2 | 1.0 | 11 | nothing |
10 | Right-click context menu | Yes (2.05 F7 popup order and rules known from the decompile; no world right-click exists) | 2 | 3 | 0.7 | 12 | nothing; open: 1.39 popup, History window contents |
07 | UDP peer channel | Yes | 1 | 3 | 0.3 | 13 | nothing (UDP is optional; TCP fallback covers it) |
05 | Player save and load | Yes (server-owned) | n/a | 0 | n/a | n/a | nothing, nothing to build |

Suggested build order (dependencies, not just score): 13, 12, 08 (opt-in), 03, 02 with 04 together, 14 (ignore first), 11, 01, 09, 06 (after the lab capture), 10, 07.

Items that truly need a new decompile or capture

1. ~~Decompile, 2.05: the death handler (topic 04).~~ **Filled** by the 2026-10-24 2.05 decompile (timings, input lock, hearts by AP, destination, no respawn packet, CLAIMPKER from the victim). Left: the "same name group" respawn-point rule.
2. ~~Decompile or capture, 2.05: the sprite-index to animation table and the Ctrl+letter emoticon mapping (topic 11).~~ **Filled**: wire sprite formula and step table, and the Ctrl+letter lookup (a local prefix match on `emoticon_<letter>`, no packet). Left: how other players see an emote, and swim on the wire (none in 2.05).
3. ~~Decompile or screenshot, 2.05: the F7 popup and right-click handler (topic 10).~~ **Filled**: item order Message, History, Buddy, Ignore, Profile; `TPersonAction` is the NPC script editor, not a player menu; no world right-click exists.
4. ~~**Lab capture, 2.22**: crossing a gmap edge in a 2x2 gmap (topic 06).~~ **Filled** (see `06-gmap-overworld.md`; 2.31 crossing also captured, see `../decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`).
5. **Still open: lab capture, 2.22, with an extended listserver stub**: add/verify/delete buddy and the playerlist Mute item (topic 14).
6. ~~**Lab captures**: the id-39 constant for 2.17 and 2.31 (topic 08).~~ **Filled**: constants captured for 2.17, 2.22 and 2.31, and the builder decoded (`../decomp-findings-2026-10-24-b/01-tamper-check-id39.md`). Nothing left to decompile; only whether a server enforces the value is unknown.
7. ~~Optional, feel only: the 2.05 sword hit box (topic 02).~~ **Filled**: the victim's client evaluates the hit (reach 0 to 4 tiles, lateral under 2, AP 100 immune). Left: when 2.05 sends HITOBJECTS in a swing and the real damage formula.

Settled index (cross-cutting items, status as of the lab batch c, 2026-10-24)

These are protocol items from the laundry triage that are no longer "needs decompile or capture". Detail and provenance are in the linked notes; nothing here is new evidence.

Item	Status	What is settled	Still open	Where
`PLI_ENTERLEVEL` (46), 2.31	**Settled** (lab, real client)	Payload is always `GCHAR 1`. Sent on every level entry (login, each gmap edge crossing, doors and edge links, server-side warps), just before the first props of the new level: in the same frame as a cached-level link request, otherwise in the first frame after the server's level answer. 2.31 does not re-send id 39 at any of these. 2.22 never sends 46 (0 of 8 transitions) but sends 39 after each client-initiated link request. Not sent by 2.17.	Script-initiated warps (`setlevel2`) not captured.	`../decomp-findings-2026-10-24-b/07-lab-findings-46-186-200.md`, `../decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`, `08-tampercheck.md`

Item	Status	What is settled	Still open	Where
`PLO_SERVERWARP` (178), 2.31	**Settled** for the client side (lab: one real chain, field-order cases synthetic)	Four comma fields: field 0 unused, 1 = name, 2 = host, 3 = port. The client closes the old connection and logs in again to field 2 and 3 with a new key and the same stored credentials; no listserver step. Fewer than four fields is ignored; swapped host and port fails to connect.	The text a real service sends is unobserved (the lab text came from the open-source listserver reimplementation). 2.22 untested.	`../decomp-findings-2026-10-24-b/08-lab-findings-levelchange-24-178.md`
`PLI_SHOWIMGPLAYER` (24) relay	**Partly settled** (lab)	2.31 sends `GCHAR 9` at login and about every 5.13 s; 2.22 sends it too, but only with another player in the level. The stock server relays any payload unchanged as `PLO_SHOWIMGPLAYER` (32) to the others in the level; injected payloads (synthetic) caused no disconnect and no visible reaction.	What the 9 means; what a receiver does with it; the exact 2.22 condition.	same note
`PLO_BOARDMODIFY2` (186), 2.22 and 2.31	**Settled** for the client reaction (synthetic frames)	Layout `c mapX, c mapY, c x, c y, c w, c h` + `w*h` GSHORT tiles. Bytes after the tiles are unread padding (a second record there is not applied). One byte short is dropped silently. A block past column 63 is clipped, not rejected. Identical on 2.22 and 2.31.	No server has been seen sending 186. `mapX`/`mapY` other than the current cell untested.	same note; `../../protocol/packet-ids.md`
2.00 listserver, client side	**Partly settled**	Client sends one plain zlib frame `GCHAR 0` + `newmain`, no account or password, then goes straight to the gserver (no second listserver connection). 2.00 accepts a 2.17-style list reply.	The reply format of a real 2.00-era listserver is **unobserved** (the lab reply was synthetic).	same note; `../decomp-findings-2026-10-24-b/04-v2.00-client.md`
Tamper check (PLI 39) derivation	**Settled**	Two chained CRC-32 words over the client's own memory image, each written as 5 bytes in a non-GINT5, unmasked form; a constant per build. 2.31 re-sends every 3.2 s; 2.22 at a gmap crossing; 2.17 once.	Whether any server enforces it.	`08-tampercheck.md`, `../decomp-findings-2026-10-24-b/01-tamper-check-id39.md`, `../../version-profiles/tampercheck-words.tsv`
`PLO_HIDENPCS` (151)	**Settled**	No payload; removes the NPCs with ids 1 to 999 of the active level (id 0 excluded). Wire format unchanged from 2.17.1 to 2.22.	none	`../decomp-findings-2026-10-24-b/02-server-to-client-layouts.md`, `../../protocol/packet-ids.md`

Item	Status	What is settled	Still open	Where
`PLO_SIGNATURE` (25) quit rule	**Settled**	Value 1 quits; value 0 quits unless the connected host string is the loopback address; any other value is ignored. The public-server value 73 is in the ignored group, so it never triggers the quit rule. Same in 2.17.1, 2.22, 2.31; the id is handled from 2.00.	none	`../decomp-findings-2026-10-24-b/05-open-items.md`, table below

Not settled, for contrast (all in `../decomp-findings-2026-10-24-b/05-open-items.md`): `PLI_SHOWIMGPLAYER` (24) payload meaning, the extra 2.31 step after `PLO_SETACTIVELEVEL` (156), client ids 36/42/45 in 2.22, the 2.31 window-title scan.

Still open after the 2026-10-24 decompile pass (new or remaining)

- How other players see a Ctrl+letter emote (topic 11): the key path sends nothing, and the captured SHOWIMG frames are unexplained. A lab capture of a real 2.05 client would settle it.
- Swim on the wire (topic 11, 04): none found in 2.05; swim is a local mode.
- Damage formula (topic 02): inferred sword power / 2, unconfirmed.
- A few unlabelled player-mode values (topics 04, 11). STATUS bit 0x20 is the `hidesword` timer (`../status-bit-0x20.md`).
- The "same name group" rule for the saved respawn point (topic 04).
- 1.39: F7 popup items, death handler and sword hit box were not read (topics 10, 04, 02).
- History window contents and the ignore auto-reply send condition (topic 10).

Tag note: `[dec]` marks a fact derived from the 2026-10-24 static analysis of a client build (behaviour only, no code); `[srv]`, `[cap]`, `[inf]` and version tags keep their earlier meaning (server source, capture, inferred).

Corrections made on 2026-10-24 from the decompile pass: topic 11 no longer says Ctrl+letter sends `PLI_SHOWIMG` (24) (only the script command `showimg` does), and topic 10 no longer lists `TPersonAction` as an unexplained player form. Facts are derived; see `../decomp-findings-2026-10-24/`.

Public-server "unhandled" ids, identified from GServer-v2 and placed

Id	Name	Direction	Meaning in one line	Topic
10	ISLEADER	S to C	you run this level's baddies and timeouts	02, 04 (07 for routing)
25	SIGNATURE	S to C	one byte, 73 = more than eight players; sent at every login. Client rule (2.17–2.31 handlers, 2026-10-24): value 1 quits, value 0 quits unless the host is the loopback address, any other value (such as 73) is ignored, so 73 never triggers the quit rule (settled, see the settled index above)	cross-cutting (below)

Id	Name	Direction	Meaning in one line	Topic
39	LEVELMODTIME	S to C	`i5` level modification time for the client cache (not the same as PLI 39)	06
41	STARTMESSAGE	S to C	server message text (HTML, one line)	12
44	HASNPCSERVER	S to C	only with the NPC-server build; the lab build does not send it	07
47	STAFFGUILDS	S to C	guild names that count as staff in the player list	10
156	SETACTIVELEVEL	S to C	level name that receives following NPC/chest/baddy packets (the gmap name on a gmap)	06
168	UNKNOWN168	S to C	blank packet the login server sends	cross-cutting
171	BIGMAP	S to C	`text,image,x,y` big map settings (2.1+)	06
172	MINIMAP	S to C	`text,image,x,y` minimap settings (2.1+)	06
174	GHOSTICON	S to C	1 = ghost players present (free accounts); GServer sends 0 per level	cross-cutting
179	RPGWINDOW	S to C	two greeting lines in quotes	12
180	STATUSLIST	S to C	player-list status names (Online, Away, ...)	14
182	LISTPROCESSES	S to C	process-list request (anti-cheat), client reply is PLI 44	08

Cross-cutting handling, all one-liners: store 25 and 168 and ignore; store the ghost flag from 174; none of these changes play. Also note `PLI 39` (client to server) is the tampercheck in the Graal_analysis notes and `PLI_MAPINFO` in the older GServer-v2 header (topic 08).

Facts worth knowing before writing code

- **GServer-v2 drops a client after the 6th unknown packet id** (`msgPLI_NULL`). Never send an id that is not in the profile's `pli` set, and never `PLI 43` (mute) to it (topic 14).
- **Most of the game is client-authoritative.** The server clips and forwards. Inventory counts, hearts (not above the server value under 40 AP), death bit and combat events all come from the client (topics 02, 03, 04).
- **Lab server facts**: built without the NPC-server flag; options in `servers/lab/config/serveroptions.txt` (bushitems, respawntime 15, apsystem on, maxplayers 128); `maps`, `gmaps`, `bigmap` and `minimap` are all empty; `servermessage.html` is the stock sample. To test a topic, edit a **copy** of the config and restart the lab gserver; use display :9 only.
- **Privacy**: the pcaps contain the lab password in plaintext. Use the cleaned decoded frames (`examples/lab_replay`) and never commit pcaps or the box-only files.
- **Snapshots**: laundry list "Done" items (warp, other players, DELPLAYER, file transfer) match later Seihai work than the older local clones; these notes use `main` at `06aacfd`.

Rules these notes follow

Specs are derived notes and are untested. GServer-v2 is described, not copied. Graal_analysis boundary: derived, cleaned notes only.

`client-internals/feature-timeline.md`

Client feature timeline, 2.00 to 2.31

Date: 2026-10-21

Source: the release notes shipped in the Graal Reborn 2.2.2.0 folder (`newfeatures2001.txt` … `newfeatures2005.txt`, summarized). The file names are release years, not version numbers: `2001` covers 2.00–2.12 (it also repeats 1.32–1.41), `2002` covers 2.13–2.17, `2003` covers 2.18–2.21, `2004` covers 2.22, and `2005` covers 2.3–2.31. The protocol column links each note to what the binaries show (`protocol/client-versions.md`). Tags: **[doc]** release notes · **[bin]** client binaries.

Version	Release-note highlights	Protocol anchor
2.00	Online-only: 2/3/4-player mode and save games removed. Bush rupees and pot hearts go server-side. `warpto <account>` for GPs.	
2.01	Level preloading on maps. `hitobjects` added as the fair replacement for `hitplayer`. `openurl2`.	`PLI/PLO_HITOBJECTS` (36/46)
2.02	Hats and signs with images, string parsing. Free-server limits on light effects and "complex" ganis.	
2.03	`triggeraction x,y,action,params`. RC and NC get `/` chat commands. The built-in zlib.dll is replaced by in-game compression code "because it has been abused".	`PLI/PLO_TRIGGERACTION` (38/48)
2.04	Engine work: gani `PLAYSOUND PARAM1`, `replacegani`, sound and animation changes.	
2.05, 2.05 rev 2	Mostly bug fixes. Server name in the window title. **2.05 rev 2 is the `GNW03081` build.**	type 0, gen 3, 59 PLO ids **[bin]**
2.1 (rev1–3)	New start window and server list with account info. New image engine without GDI. Gani scripts (`SCRIPT … SCRIPTEND`). String arrays. `play2`. Auto-mapping. **Big map and minimap set by the gserver (`bigmap=` / `minimap=` in serveroptions) instead of the listserver. Ghost mode** for free players, queued until activated. Alt+0 log window. `graalversion`.	`PLO_BIGMAP` / `MINIMAP` (171/172) and ghost ids (170, 173, 174) are handled by 2.17 but not 2.05 **[bin]**
2.12 (rev1)	Graal2001/Graal2002 client compatibility. Multi-level editing. `wraptext`, `wraptext2`.	
2.13 (rev3–4)	Editor control-NPCs. **New RC with file browser, server list, and RC+NC in one program.**	`RemoteControl.exe` (see `tools/remotecontrol.md`)
2.14 (rev4–7)	**Server list gets two tabs** (company servers, classic and playerworlds). Voice chat. **Player-list status icons** (`playerlisticons=` in serveroptions). **`serverwarp <world>`. Files over 64 KB can be downloaded** (`downloadpos`). Gmaps auto-download.	`PLO_STATUSLIST` (180), `SERVERWARP` (178), `LARGEFILESTART/END` (68/69), all handled from 2.17 **[bin]**

Version	Release-note highlights	Protocol anchor
2.15 (rev2)	Bug fixes, "enabling Graal2002". `showimg2`.	
2.16 (rev1–7)	Real transparency (`seteffectmode`). Gani attributes `#P10`–`#P30`. More server-list features. UDP check improved, with a warning when UDP is blocked.	`PLPROP_GATTRIB10-30` (54–74)
2.17 (rev1)	Graal Kingdoms speedups. **Level heights stored in level files instead of the gmap.**	`PLO_BOARDHEIGHTS` (185) **[bin]**; still type 0, gen 3; adds `PLI_TAMPERCHECK` (39) and the signed `arrays2.dat` listserver config **[bin]**
2.18 (rev1)	**"New password encryption"** for stored passwords. Preloaded levels on entering a server.	
2.19 (rev1)	Released alongside v3. New start screen and server list. **"Better compression"** (bz2). `this.` strings on the client. On NPC-server servers the client stops sending non-`client.` strings.	gs2lib: `GNW15053`, type 4 / gen 4 (bz2 + partial XOR) **[srv]**
2.2	Multi-byte sign text (Korean, Chinese, Japanese). **Server-side tile changes on gmaps via `updateboard2`.**	`PLO_BOARDMODIFY2` (186), handled from 2.22 **[bin]**
2.21	**"Network traffic reduced drastically"**. `triggeraction 0,0,clientside,…` reaches a weapon's client-side part.	gs2lib: `GNW01113`, the compression byte **[srv]**
2.22	**Third server-list tab** for under-construction playerworlds (gold/VIP only). `gravity`. `downloadsize`.	type 5, gen 5, `GNW03014`, listserver `GNW30123` **[bin]**
2.3	**Editor and online client split into separate exes.** `FILENAMECACHE.txt`. New timer (no 99% CPU). B-tree NPC lookup. **Classic client-side mode removed: Classic moved to the NPC-Server.** Graal3D server tab. `graal://` URL handler with `serverstartparams`. **Full-precision player and NPC positions** ("enabled once all clients are switched"). `timevar2`. More hacking checks.	`PLO_HASNPCSERVER` (44) dropped, `PLO_MOVE2` (189) and `LARGEFILESIZE` (84) added in 2.31 **[bin]**
2.31	Float-to-string fix, keyboard config saved, gmap NPC border crossing fixed.	`GNW28015` **[bin]**

Reading the timeline against the binaries

- Several features announced as "will be enabled later" show up in a client's receive switch one or more versions before the server used them (full-precision moves in 2.3, level heights in 2.17). A handled id is therefore not proof that servers of that era sent it. **[bin][inf]**
- The jump from 79 to 81 handled ids between 2.17 and 2.31 is small. Most 2.19–2.31 changes are cipher, compression, and scripting changes, not new packets. **[bin]**

`client-internals/foot-test.md`

Foot test (movement collision)

Date: 2026-10-14

Superseded. This note records the single-point wall test recovered from the SDL reimplementation (2dadventureclient). The real clients (2.05 through 2.31) use a six-probe body-box routine with corner nudges and

flush snaps. See Seihai `documentation/walking-collision.md` for the current description, and `archival/foot-test-sdl.md` for the preserved SDL snapshot.

The original content is retained below for reference.

Derived from the SDL reimplementation of the client movement loop and the type table. No decompile material.

Single-point test

Arrow keys set a pixel step of `16 * 0.30` and a sprite direction (0 up, 1 left, 2 down, 3 right). The wall test is a single point, not a box or a sweep:

```
testx = (pixelX + moveX + 32) / 16
testy = (pixelY + moveY + 44) / 16
```

If `isOnWall(testx, testy)` is false, the new position is accepted. If true, the step is rejected. There is no slide, no corner resolution, and no water slowdown in this loop. `isOnWater` exists but is not consulted by the keyboard path.

Predicates

- Outside the board on x or y: wall (edge is solid until a LINK says otherwise).
- Inside: type byte at `levelTiles[round(y)*64 + round(x)]` ≥ 20 is wall; type == 11 is water (does not block).
- Chair, bed, swamp, and lava types are stored and ignored by this path. Type 12 (old note: lava) is walkable here.

Stamp consequence

The 2×2 Bush stamp writes ids 2, 3, 18, 19. Those type bytes are 20/22, so the bush is wall. The stamp file itself carries no collision column; the foot test decides after the ids are written.

Official ~2.05 used a different body-block mask and type-0 allow lists. The measured path above keeps only the two predicates.

`client-internals/gameboy-demo.md`

Graal Game Boy Color demo (1999)

Date: 2026-10-29, emulator run added 2026-10-30

This note covers the full disassembly of `Graal V1.1 Adventure Demo (PD) [C].gbc` (262,144 bytes, MD5 2b2fef91…, the same bytes as `graal.gb` in `graalgb.rar`). The first look at this ROM is in `precursors-and-fan-builds.md`.

The disassembly itself is in the project's shared folder under `graal-decompile/gameboy-demo/`, not in this repo. That folder holds RGBDS-syntax sources for all 16 banks, the graphics as `.2bpp` files, PNG renders of every room, the scripts that produce them, and a README. The sources reassemble to a byte-identical ROM. That was checked with a small assembler written for this job; RGBDS itself was not run. This note is cleaned: it has no code and no code addresses. ROM locations are given only where they describe a data format. The first

sections come from reading the code. The section "Seen in an emulator" adds what the running ROM did, and corrects the static read where they differ.

What it is built with

- **GBDK 2.x and its LCC C compiler.** The start-up code is GBDK's crt0: interrupt handler chains, the `.mode` table, `add_VBL` and the rest. The game code has LCC's fingerprints: arguments on the stack, `switch` statements as jump tables, and `int` constants read from a per-bank constant pool. Of 161 functions, 41 are GBDK library or crt0 routines (`set_bkg_data`, `set_win_tiles`, `set_sprite_palette`, `joypad`, `memcpy`, `arand`, the multiply and divide helpers and so on). The other 120 are the game.
- **Everything runs in the VBlank interrupt.** `main` sets up the screen, loads room 0, installs one VBlank handler with `add_VBL`, turns on sound, switches the CGB to double speed and returns. From then on crt0 just idles. The handler does one game tick: OAM DMA, scrolling, the window slide and music on every VBlank, and the game logic on every second one (about 30 Hz).
- **Banking.** One helper writes the MBC1 bank register and remembers the bank. Code in banks 1 and 3 is reached through small bank-0 wrappers that switch, call and switch back.

Header and bank map

The cartridge type is `$02` (MBC1+RAM), but the RAM size byte is 0. The CGB flag is `$80`, so the ROM runs on both models. Only banks 0 to 5 hold anything. Banks 6 to 15 are all `$FF`.

Bank	Contents
0	crt0, the GBDK library, and the core game: room loader, input, text box, Select menu, collision, the VBlank tick
1	Entity update and AI, entity drawing, lifting and throwing, camera, the music player and its note table
2	Graphics: two tilesets of 256 tiles, the 82-tile font, walk bitmaps, five BG palette sets, animated tiles, 172 sprite tiles
3	Objects (lifted and thrown things), sound effects, entity set-up, sprite sorting, two AI routines, the character table with heads, palettes and names
4	Room pointer table and rooms 0 to 4
5	Rooms 5 and 6

Screen and coordinates

- A room is exactly one 32×32-tile BG map, which is 256×256 pixels. Nothing is streamed: the loader writes the whole map to VRAM, and the camera scrolls over it.
- Positions are 16-bit values in half pixels. A map tile is 16 units (8 pixels).
- Directions are numbered 0 up, 1 left, 2 down, 3 right. That is the same order as the PC client.
- The window layer holds the text box and the Select menu, and slides up from the bottom of the screen.
- Sprites are 8×16. The game builds them in a WRAM buffer, sorts them by Y, and copies them to the DMA page every tick.

Room format

Bank 4 starts with a table of seven 16-bit header pointers. Rooms 0 to 4 are in bank 4 and rooms 5 and 6 in bank 5; the loader picks the bank as room / 5 + 4.

Header (13 bytes):

Offset	Size	Field
0	2	map pointer
2	1	tileset (0 outdoor, 1 indoor)
3	1	BG palette set (0 to 4)
4	1 + 2	link count, link list pointer
7	1 + 2	text count, text list pointer
10	1 + 2	NPC count, NPC list pointer

- **Map:** 1,024 tile ids, then 1,024 CGB attribute bytes (palette, VRAM bank, flips), both 32×32. The loader keeps two WRAM copies of the tile ids: one as loaded and one that changes when things are lifted. Collision reads the second one.
- **Link (8 bytes):** x, y, width, height, new x, new y, then the target room as a 16-bit number. A new x or y of −1 keeps the player's coordinate. This is the PC client's link record (`file x y w h newx newy`), with a room number in place of the file name. The player's tile is tested against the rectangle on every step.
- **Text (4 bytes):** x, y and a pointer to a zero-terminated string. A text at (0, 0) is never read from the map; NPCs and events show it by index. Signs are read only while the player faces up and the tile above is solid.
- **NPC (16 bytes):** x and y in tiles, direction, character type, two unknown bytes, a text index (−1 for none), five movement bytes (a mode, then patrol bounds), a 16-bit flag word whose eleven low bits are copied into separate entity fields, and a 16-bit event number. Flag bit 0 makes the NPC turn to face the player and chase once lined up. The other flags are still unknown.
- **Event 1** is the only event in the demo, on the captive in room 4. It is a timed script that counts down a timer in the entity. The captive walks a few steps, stops, lifts the stone in front of it, turns and throws it, and then text 1 opens ("Now press Select…"). While it runs, the player is held in the lower part of the room.
- **Walk bits:** each tileset has a 256-bit walkable bitmap in bank 2, plus a second 256-bit set that the object code reads when something is put down. What that second set means is not settled.

The seven rooms

The six outdoor rooms form a 2×3 grid. Room 1 is the inside of the house in room 6.

```
room 4 (captive)   room 6 (house, "That's your house!")
      |                    |
room 3 (two chasers) room 5 ("Hello!")
      |                    |
room 0 (start) ---- room 2
```

Room	Tileset	Links to	NPCs (character type)	Texts
0	outdoor	2 (east), 3 (north)	–	a sign: "Welcome to Graal! Demo version by Stefan Knorr"
1	indoor	6 (the door)	–	–
2	outdoor	0 (west), 5 (north)	Ryu	–
3	outdoor	4 (north), 0 (south)	Pach Soldier, Squall	–
4	outdoor	3 (south)	Fung Si Yun (the captive), event 1	the captive's two texts
5	outdoor	2 (south), 6 (north)	Mandi, Lunar, Mandi	"Hello!" (all three NPCs)
6	outdoor	5 (south), 1 (the house door)	Vincent, Squall	a sign: "That's your house!"

The "Welcome to Graal" text is an ordinary sign in the first room, not a title screen. There is no title screen: `main` goes straight to room 0.

Room 4's texts, as the 18-character lines lay them out:

> Thanks for freeing me! I will follow you and help you completing your task. But first let me show how to lift stones! Press A to lift, A again to throw!
>
> Now press "Select" and switch to "Fung Si Yun"

PNG renders of all seven rooms with their real palettes, and of the whole grid, are in the shared folder under `png/`.

Text

- The text box is three lines of 18 characters at the bottom of the screen. The strings are padded with spaces so that words break at the right places.
- A new press on the d-pad while the box is open shows the next three lines. After the last page the box closes. In play, only a direction other than the one the player faces turns the page; pressing the facing direction does nothing.
- Character codes: `0x20` space, `0x21`–`0x3A` A–Z, `0x3B`–`0x54` a–z, `0x55`–`0x5E` 0–9, then punctuation (`0x5F` "!", `0x62` ".", `0x63` ",", `0x67` used both as an apostrophe and as a quotation mark). The tile drawn is the code minus `0x20`; the font sits at tile 0 of CGB VRAM bank 1.

Characters

Bank 3 holds a table of 31 character types. Each type has a name, three CGB OBJ palettes and a 16×16 head with two frames. Several types reuse another type's head with new colours.

#	Name	#	Name	#	Name	#	Name
0	Fung Si Yun	8	Zoraas	16	Oldman	24	Green Pach
1	Pach Soldier	9	Mad Max	17	Gold Soldier	25	Blue Lizard
2	Ninji	10	Pachukite	18	Gold Squall	26	Fire
3	Lunar	11	Matt	19	Gold Zoraas	27	Silver
4	Squall	12	Brandon	20	Gold Max	28	Theif (sic)
5	Vincent	13	Bob	21	Gold Matt	29	Red Squall
6	Mandi	14	Athena	22	Gold Silver	30	Water Athena
7	Ryu	15	Lizardon	23	Red Max		

Recoloured heads: Gold Soldier and Green Pach use Pach Soldier's head; Gold Squall and Red Squall use Squall's; Gold Zoraas uses Zoraas's; Gold Max and Red Max use Mad Max's; Gold Matt uses Matt's; Blue Lizard uses Lizardon's; Water Athena uses Athena's; Gold Silver uses Silver's.

These names are the character types of the demo, not only baddies. NPCs in rooms pick one by number, and so does the player.

- The player starts as type 0. Fung Si Yun, the captive in room 4, is type 0 too.
- **Select** opens a carousel in the window that shows three heads and the name of the middle one. Left and right turn it, and each step changes the player's own character type by one, wrapping through all 31. So the Select menu, as coded, is a free character picker over every type. It is not limited to the companion that the room 4 text mentions.

Entities

- There are at most eight entities. Entity 0 is the player and the rest come from the room's NPC list. Each one is 50 bytes.
- Fields that are understood: position, character type, direction, text index, event number, state, animation frame, OBJ palette, movement mode and bounds, a timer, a knock-back velocity, and the id of a carried object (−1 for none).
- States: 0 idle, 1 walking, 2 reading a text, 3 swinging the sword, 4 knocked back, 5 chasing the player, 8 carrying, 9 lifting. The update routine dispatches on the state. Movement mode 1 is a random walk; modes 2 to 4 patrol inside bounds kept in the record, and their exact rules were not worked out.

Player controls

Button	Effect
D-pad	Walk; turn the page in a text box (see below); turn the Select carousel
A	The character's A item: lift and throw for Fung Si Yun, a bomb for Ninji, a boomerang for Ryu, nothing for the rest (see below)
B	Sword, unless carrying. Every character has one; only its colour changes
Select	Open or close the character carousel
Start	Toggle music

Lifting works on 2×2 map objects. The game looks two tiles ahead and compares the object's top two tile ids with a list of four liftable kinds: three kinds of bush and stone, and a dropped object. For the first three it writes that kind's "lifted" 2×2 tiles into the map; for a dropped object it puts back the room's original tiles. The player goes into the lifting state, then carries the object as a sprite. A thrown object is tracked as an object of its own. In play it always broke where it came down, and nothing was written back into the map (see below). The "dropped object" kind in the list was never seen.

Sound

- There are eight one-shot sound effects on the two pulse channels and the noise channel. They are skipped while music plays.
- A music player is in bank 1, with a 72-entry table of note periods. **Start** turns it on and off.
- The player enables cartridge RAM before it reads, and its one song pointer is `$A000`, the start of cartridge RAM. The header declares no RAM, and nothing in the ROM writes to that area. So the demo carries no song of its own, and what Start plays depends on whatever answers at `$A000`. This is a static reading. It may be a leftover from a development cartridge that had a song loaded into its RAM.

Seen in an emulator

The ROM was run in PyBoy 2.8.1 in CGB mode, headless, with scripted input, screenshots every one or two frames, and the entity records read from WRAM while it ran. Contact sheets of every test, and the scripts that made them, are in the shared folder under `graal-decompile/gameboy-demo/emulator/`. Times are in 60 Hz frames. The game logic runs every second frame, so one logic tick is 2 frames.

Start and screen

- The Nintendo logo is followed straight away by room 0. There is no title, no HUD and no health display at any time.
- The CGB really is put into double speed.

- The camera keeps the player in the middle of the screen and stops at the room edges.

Walking

- The player moves 4 pixels per logic tick, which is 2 pixels a frame or 120 pixels a second. A room (256 pixels) takes about 2 seconds to cross.
- **Against a wall** the player keeps facing the wall and the walking animation keeps running in place. There is no push pose, however long the button is held.

Text

- Walking up into a sign (the stone post in room 0) or into an NPC with a text opens the box. It slides up from the bottom over a few frames.
- Capital letters are drawn in red and everything else in black.
- Select still opens the character menu while a text box is up.

Sword

- B swings the sword in an arc from one side to the front of the facing direction. One swing takes about 24 frames. Holding B swings again and again.
- A hit on an NPC knocks it a few pixels away, and it blinks for a moment. Nothing dies.
- The soldiers in room 3 chase the player and swing back. Their hits knock the player back the same way. Pinned against the top of the room, the player was knocked back over and over and could not get away, but nothing else happens: there is no damage and no game over.

Lifting, carrying and throwing (Fung Si Yun only)

- A in front of a bush, the stone post or another liftable 2×2 object swaps it for its "lifted" tiles at once (a dark stump for a bush). The object is drawn as a sprite where it stood for 10 ticks (20 frames), and then appears over the player's head.
- **While carrying**, the player is drawn facing the screen with both arms up, whatever the walking direction, and the object covers the player's head completely. So the carried object hides the head; it does not hide the whole player.
- A throws the object in the facing direction. It flies for about 6 ticks, then breaks into four leaf pieces that hang for about 8 ticks and vanish. This happened both when it hit a tree and on open grass. Nothing was put back into the map.
- Only character type 0 can lift. With any other type, A next to a bush did nothing.

Respawn

- Lifted bushes do not grow back while you stay in the room. One was still a stump after two minutes.
- Leaving the room and coming back restores every bush and stone, because the room is reloaded from ROM.

The character menu and A items

- The menu fills the whole window. It shows three heads with arrows either side, the middle character's name with capitals in red, and "Button B" and "Button A" with an item icon under each.
- Every character has a sword under B, in one of three colours (red for most, blue for Pach Soldier and Gold Soldier, purple for Mandi).
- Only three characters have anything under A, and the icon is chosen in code by the character number:
    - **Fung Si Yun (0):** a bush, meaning lift and throw.

    - **Ninji (2):** a bomb. A puts it on the ground in front. It sits for about 110 frames, flashes for about 70 more, and then blows up as a large plus-shaped blast about three tiles each way. The screen jumped as the blast went off, which looks like the blast knocking the Ninji player back; that was not checked in the entity record.
    - **Ryu (7):** a boomerang. It flies about four tiles in the facing direction and comes back to the player, about one second in all.
- This is more than the static read found. The A items are a second action set, picked by character, and the room 4 text ("switch to Fung Si Yun") is about getting the lift action.

Rooms and NPCs

- All seven rooms loaded and looked like the PNG renders made from the ROM.
- Room 2: Ryu stands still. Room 5: Mandi, Lunar and Mandi stand still; by the room data, each says "Hello!".
- Room 3: the first soldier patrols left and right along the top of the room at 1 pixel a frame, half the player's speed. The second wanders. Once lined up with the player, both chase at a little over 1 pixel a frame and swing their swords.
- Room 6: Vincent and Squall wander around the house.
- **Room 4, the captive.** Walking into Fung Si Yun opens the first text. When it closes, he walks two tiles left, lifts the bush there, turns and throws it, and the second text opens ("Now press Select and switch to Fung Si Yun"). This matches the event script in the code. The player and the captive look the same, because the player also starts as Fung Si Yun. The text reads as if the player was meant to start as someone else.

Sound

- The sound effects play (a noise burst for the sword).
- **Start silences the game.** With nothing at the song address, reading it gives `$FF` (as on a cartridge without RAM). The music player then writes 0 to the master volume and the channel routing, and Start again does not undo it. From then on the sword and every other effect are silent until the game is reset. How `$FF` is read here depends on the emulator, but it is also what a real cartridge without RAM usually returns.

Compared with the PC client

- The same link record: a rectangle plus a new x and y, with −1 to keep a coordinate.
- The same direction numbers, and the same lift, carry and throw of 2×2 bushes and stones with A. Here the liftable objects are a fixed list of tile pairs.
- Rooms are fixed 32×32-tile maps, not 64×64 levels. Positions are in half pixels, and there is no NPC scripting: NPC behaviour is a few built-in movement modes, flags and event numbers.
- The text alphabet is a straight ASCII-like run, not the PC sign alphabet.

What a Seihai "Game Boy demo" profile would need

This is the look and feel the run showed, as a list for a profile that copies the demo.

- One 256×256-pixel room at a time, with no scrolling between rooms: a link loads the next room with the screen off.
- Movement at 2 pixels a frame (logic at 30 Hz, 4 pixels a tick). No push pose against walls; the walk cycle keeps running.
- A text box of three lines of 18 characters in the window, sliding up, capitals in red. Paging by a d-pad direction other than the one faced.

- No HUD, no health, no death. Hits only knock back.
- Sword on B for everyone. A depends on the character: lift and throw, bomb, boomerang, or nothing.
- Carried objects cover the head, and the carrier is drawn facing the screen.
- Thrown objects always break. Nothing respawns until the room is loaded again.
- A Select menu that picks any of the 31 characters, with their head, name and B/A icons.
- No chat and no network. There is no chat bubble in the demo.

Open

- The music data format, since no song is in the ROM. Pressing Start mutes the game rather than playing anything.
- Several entity fields and most NPC flag bits.
- The exact patrol rules of movement modes 2 to 4. The run showed a left-right patrol and a wander, not the rules behind them.
- Whether the Select carousel was meant to stay a free picker in a later version.
- The "dropped object" kind in the liftable list, which was never seen in play.